

MINISTRY OF EDUCATION AND TRAINING
UNIVERSITY OF ECONOMICS AND FINANCE
HO CHI MINH CITY



FINAL PROJECT REPORT

Games Development

TOPIC: MOUNTAIN KIN: THE KING'S DEMAND

Class: 251.ITE1279E.B02E

Instructor: **MSc. Nguyễn Thanh Điền**

Performed by Student:

Nguyễn Ngọc Triệu	22D1ITE-SWE02	225054127
Hoàng Văn Thiên	22D1ITE-SWE03	225051915
Nguyễn Thành Đạt	22D1ITE-SWE03	225050896
Nguyễn Hoàng Phúc	22D1ITE-SWE02	225050646
Nguyễn Tấn Thiện Huy	22D1ITE-SWE02	225051460
Phạm Quang Vũ	22D1ITE-SWE02	225054138

Ho Chi Minh City, January 2026

ACKNOWLEDGEMENT

We would like to express our sincere appreciation to our instructor, **Mr. Nguyễn Thanh Điền**, for his continuous guidance, valuable feedback, and dedicated support throughout the *Games Development* course. His clear explanations, practical insights, and constructive suggestions greatly assisted our team in understanding core game development concepts and applying them effectively during the implementation of this project. The instructor's guidance helped us approach the project in a structured and professional manner, from initial idea development and game design to implementation and testing.

We would also like to thank the **University of Economics and Finance Ho Chi Minh City (UEF)** for providing a supportive academic environment and sufficient learning resources. The facilities, learning materials, and technical infrastructure offered by the university created favorable conditions for our team to research, design, and complete the game project in alignment with the course objectives.

In addition, we sincerely appreciate the cooperation and efforts of all team members involved in this project. Throughout the development process, team members actively collaborated, shared responsibilities, and supported one another in solving technical and design-related challenges. Although the workload and task complexity varied among members, each contribution played an important role in the successful completion of the game. This experience helped us improve not only our technical skills but also our communication, teamwork, and time management abilities.

Despite the limitations caused by time constraints and the academic scope of the course, we believe that this project reflects our learning progress and practical understanding of game development principles. The knowledge and experience gained from this project will serve as a valuable foundation for our future studies and professional careers in the field of game development and software engineering.

Thank you very much!

LIST OF CONTENTS

LIST OF FIGURES	iii
LIST OF TABLES.....	v
CHAPTER 1. INTRODUCTION.....	1
1.1. Background and problem statement.....	1
1.1.1. Background.....	1
1.1.2. Problem Statement.....	1
1.2. Objectives.....	1
1.3. Scope and target audience.....	2
1.3.1. Target audience.....	2
1.3.2. Project scope.....	2
1.4. Limitations.....	3
1.5. Project significance.....	3
CHAPTER 2. LITERATURE REVVIEW AND REQUIREMENTS.....	4
2.1. Technologies used.....	4
2.1.1. Game Architecture & Core Systems.....	4
2.1.2. UI & Visualization Tools.....	5
2.1.3. Development tools and version control.....	7
2.2. Audacity or other audio tools:.....	9
2.3. Requirement analysis.....	10
2.3.1. Functional requirements.....	10
2.3.2. Non functional requirements.....	11
CHAPTER 3. SYSTEM DESIGN.....	13
3.1. Character Control & Movement.....	13
3.1.1. Main Character Assets.....	13
3.1.2. Enemy Assets.....	13
3.1.3. Boss Assets.....	14
3.1.4. Player System.....	15
3.1.5. Player Component Configuration.....	16
3.1.6. Combat Feedback & Audio Integration.....	18
3.1.7. Survival & Recovery Mechanics.....	20
3.1.8. Character Stats & Posture System.....	22

3.1.9. Advanced Combat Mechanics & Target Lock System.....	24
3.1.10. Combat Data Architecture.....	26
3.1.11. Input Mapping Configuration.....	28
3.2. Collision Detection & Interaction.....	30
3.3. Score System Design.....	32
3.4. User Interface (UI) Design.....	33
3.5. Game Progress Storage.....	36
CHAPTER 4. IMPLEMENTATION AND RESULTS.....	38
4.1. Development environment and tools.....	38
4.2. Step-by-Step installation guide.....	38
4.3. Key code snippets and explanation.....	39
4.3.1. Implementing input handling and character movement control.....	39
4.3.2. Boss Artificial Intelligence and Combat Logic.....	46
4.3.3. Resurrection and Vitality Management System.....	49
4.3.4. Weapon Hit Detection and Damage Application Logic.....	52
4.3.5. Save game.....	54
4.4. Screenshots of main features.....	55
4.5. Conclusion and future work.....	56
4.5.1. Achievements.....	57
4.5.2. Limitations.....	57
4.5.3. Suggestions for future enhancements.....	57
APPENDIX.....	59
REFERENCES.....	60
TASK DIVISION AND CONTRIBUTION SUMARY.....	61

LIST OF FIGURES

Figure 2.1-1: The hierarchical game architecture design used in Mountain Kin.....	5
Figure 2.1-2: Unity Game Engine.....	5
Figure 2.1-3: Unity Asset Store.....	6
Figure 2.1-4: Unity Engine and Microsoft Visual Studio 2022.....	7
Figure 2.1-5: Unity Hub.....	8
Figure 2.1-6: Unity DevOps.....	8
Figure 2.2-1: Audacity.....	10
Figure 2.2-2: CapCut.....	10
Figure 3.1-1: The main character model used in the game.....	13
Figure 3.1-2: Enemy character model (Armored Knight).....	14
Figure 3.1-3: Boss character model.....	15
Figure 3.1-4: Player Animator Controller State Machine.....	16
Figure 3.1-5: Player Inspector configuration showing Movement and Combat scripts..	17
Figure 3.1-6: Audio Library and Stagger Handler configuration.....	19
Figure 3.1-7: Healing and Revival configuration settings.....	21
Figure 3.1-8: Character Stats and Posture configuration (SekiroStats).....	23
Figure 3.1-9: Target Lock System and Combat Effects configuration.....	25
Figure 3.1-10: Combat Script utilizing Scriptable Objects for attack data.....	27
Figure 3.1-11: PC-Based Keyboard and Mouse Input Configuration.....	28
Figure 3.1-12: Technical Implementation: Unity Input System.....	29
Figure 3.1-13: Collision-Based Defensive Interaction.....	30
Figure 3.2-1: Parry System Mechanics.....	31
Figure 3.3-1: Combat Result and Score Evaluation Interface.....	32
Figure 3.4-1: Unblockable Mechanic: Lightning Counter Tutorial.....	33
Figure 3.4-2: Parry Mechanic Tutorial Interface.....	34
Figure 3.4-3: Menu Interface Design.....	34
Figure 3.4-4: Game Over Screen Design.....	35
Figure 3.4-5: Main Menu Interface Design.....	36
Figure 3.5-1: Registry-Based Game Progress Storage.....	37
Figure 4.3-1: Boss jump function.....	42
Figure 4.3-2: Saving and Loading Game State Using PlayerPrefs.....	54

Figure 4.4-1: Mainmenu.....	55
Figure 4.4-2: Player Movement and Controls.....	55
Figure 4.4-3: Game Over Screen.....	55
Figure 4.4-4: Enemy Detection System.....	56
Figure 4.4-5: Player Healing System.....	56
Figure 4.4-6: Saving/Loading states Screen.....	56

LIST OF TABLES

No table of figures entries found.

CHAPTER 1. INTRODUCTION

1.1. Background and problem statement

1.1.1. Background

In recent years, players have shown increasing interest in challenging, mastery-driven games that reward skill, patience, and learning rather than simple progression. However, many modern games focus heavily on accessibility and fast rewards, which can reduce the sense of achievement for players seeking deeper engagement. Mountain kin: The King's Demand is designed to address this gap by delivering a focused, immersive experience where success is earned through understanding enemy behavior, precise timing, and strategic decision-making.

1.1.2. Problem Statement

Many modern action games prioritize accessibility, simplified mechanics, and rapid progression in order to appeal to a broad audience. While this approach increases player reach, it often results in shallow gameplay experiences that lack meaningful challenge and long-term engagement. As a consequence, players who seek mastery-driven combat, strategic depth, and a strong sense of achievement are underserved, with limited options that truly reward skill, precision, and learning from failure.

To solve this issue, Mountain Kin: The King's Demand is developed as a single-player, Sekiro-like action game that focuses on deliberate combat design and player skill progression. The project aims to resolve the gap between accessibility and challenge by offering fair but demanding gameplay, where success depends on understanding enemy behavior, precise timing, and tactical decision-making. Through this approach, the game provides not only entertainment but also a deeply rewarding experience that reinforces perseverance, discipline, and mastery.

1.2. Objectives

The primary objective of this project is to develop a single-player action game, Mountain Kin: The King's Demand, using Unity as the game engine and C# as the core programming language. The project focuses on creating a responsive and skill-based combat system where player performance is determined by timing, awareness, and strategic decision-making rather than automated assistance.

In addition, the project aims to design well-structured enemy encounters, coherent level progression, and immersive environmental elements that support both gameplay flow and narrative delivery. From an academic perspective, this project serves as a practical application of game development principles, including gameplay programming in C#, real-time interaction handling, difficulty balancing, and system integration within Unity, resulting in a functional and polished game prototype. Additionally, the project integrates a scoring system, a user-friendly interface (UI), and accurate collision detection to support player feedback, balanced combat, and consistent gameplay behavior.

1.3. Scope and target audience

1.3.1. Target audience

Mountain Kin: The King's Demand is aimed at players who enjoy experiences that demand concentration, persistence, and thoughtful execution. The game is suitable for individuals who prefer overcoming obstacles through observation, pattern recognition, and steady improvement, rather than relying on guided systems or instant rewards.

Additionally, the project is relevant to students and aspiring developers who wish to explore the development of action-oriented games within a modern engine environment. By working with Unity and C#, the game provides insight into interactive system construction, character behavior design, and structured level development, making it valuable as both a playable experience and an academic reference.

1.3.2. Project scope

This project involves the development of a playable single-player action game prototype that demonstrates core interaction, progression, and player feedback within a controlled game environment. The work includes the design and implementation of character actions, opponent encounters, basic progression flow, and on-screen feedback elements that support user interaction and gameplay clarity.

The project also covers the construction of a contained level structure that guides the player through a sequence of encounters while introducing environmental elements that influence movement and decision-making. Narrative context is delivered implicitly through the game world and gameplay events rather than explicit storytelling. The outcome of the project is a cohesive prototype that showcases fundamental action game design and system integration within a defined development boundary. Specifically, the game is developed to

run on the Windows PC platform (Windows 10/11) and is optimized for keyboard and mouse input.

1.4. Limitations

Due to time and resource constraints, several features fall outside the project's limitations. These include multiplayer modes, large open-world environments, advanced artificial intelligence, cinematic storytelling, and extensive content variety. Visual assets and audio elements are implemented at a basic level to support gameplay functionality. As a result, the project prioritizes technical feasibility and design clarity over graphical complexity and content scale.

1.5. Project significance

Despite its limited scale, this project holds meaningful value in both practical and academic contexts. From a practical perspective, it demonstrates the feasibility of designing a focused action game experience that emphasizes intentional interaction, structured progression, and player feedback. The project showcases how core gameplay systems can be integrated cohesively to form a complete and engaging prototype.

From an academic standpoint, the project reflects the team's ability to apply game development principles, system design, and interactive logic within a modern game engine environment. It provides hands-on experience with gameplay programming, system integration, and iterative design, serving as a strong foundation for understanding game development workflows and interactive software design.

CHAPTER 2. LITERATURE REVRIEW AND REQUIREMENTS

2.1. Technologies used

The selection of an appropriate technology stack plays a crucial role in ensuring the stability, performance, and extensibility of the game project. This section presents the primary tools and technologies utilized in the development of Mountain Kin: The King's Demand.

Unity Game Engine is employed as the core development platform due to its robust support for real-time interaction, physics simulation, and scene management. It provides an efficient environment for rapid prototyping and iterative development, enabling the integration of gameplay logic, visual elements, and user interaction within a unified framework.

C# Programming Language is used to implement gameplay logic, character behavior, and system interactions. Its object-oriented structure supports modular design and code maintainability, allowing game systems such as player control, enemy logic, and state management to be developed and tested independently.

In addition, built-in Unity tools such as the Animator, Physics system, and UI framework are utilized to manage animations, collision detection, and on-screen feedback. Together, these technologies form a cohesive development environment that supports the creation of a functional and well-structured action game prototype.

2.1.1. *Game Architecture & Core Systems*

The internal logic of Mountain Kin: The King's Demand is designed following a hierarchical game architecture that clearly separates high-level management, core gameplay systems, and low-level entity behavior. At the top level, the game architecture is governed by a central game manager responsible for coordinating game states, scene transitions, and overall game flow. This ensures that global rules and progression are consistently maintained throughout the gameplay experience.

At the system level, core gameplay modules such as player control, enemy AI, combat handling, and resource management are implemented as independent yet interconnected subsystems. Each subsystem focuses on a specific responsibility—for example, the player control system manages input processing and stamina usage, while the enemy AI system handles state transitions, pathfinding, and attack behaviors. These systems communicate

through clearly defined interfaces and event-driven mechanisms, allowing them to interact without tight coupling.

At the entity level, object-oriented programming principles are applied to encapsulate character attributes and behaviors within dedicated classes, including health management, stamina consumption, skill execution, and enemy states. Real-time events such as player actions, collisions, and combat outcomes are processed through an event-handling layer, enabling responsive and dynamic gameplay. By organizing the internal logic according to this hierarchical architecture, the game achieves a clear separation between gameplay logic, visual presentation, and user interaction, supporting stable performance, maintainability, and future scalability.

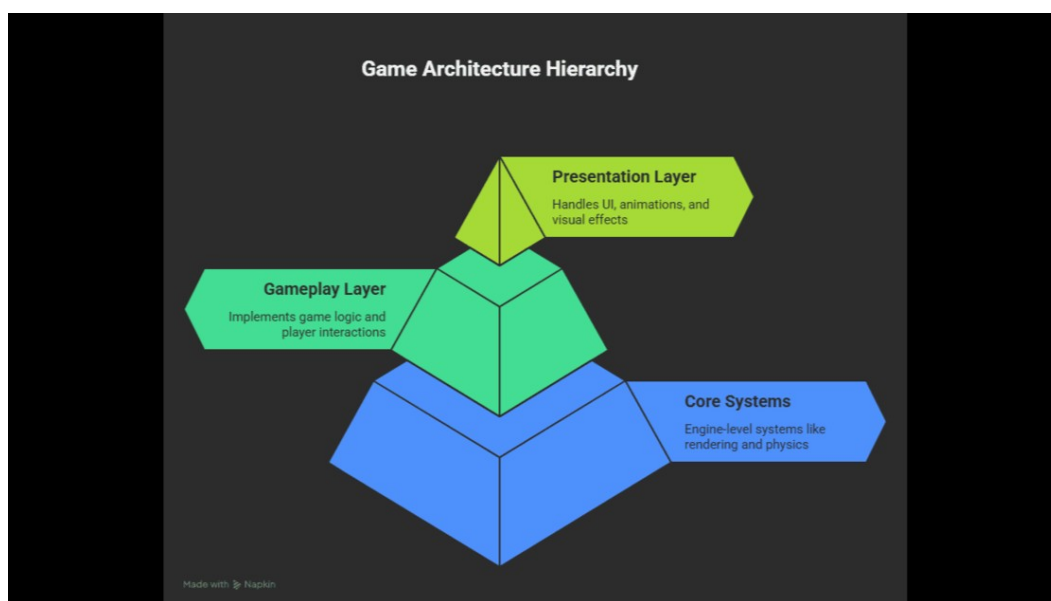


Figure Error: Reference source not found-1: The hierarchical game architecture design used in Mountain Kin.

2.1.2. UI & Visualization Tools



Figure Error: Reference source not found-2: Unity Game Engine

The user interface of the system is developed using Unity as the primary front-end platform, providing an interactive and immersive environment for users. Unity is chosen for its powerful rendering capabilities, flexible UI system, and cross-platform support, allowing the application to deliver a consistent user experience across different devices. The interface is implemented using Unity's built-in UI tools, including Canvas, UI elements, and event systems, which enable responsive menus, real-time interaction, and smooth transitions between game scenes. User inputs such as movement, combat actions, and menu navigation are processed in real time, ensuring immediate feedback and intuitive control. By leveraging Unity as the front-end framework, the project achieves a visually engaging interface that seamlessly integrates user interaction with core gameplay logic.

In addition to engine-based tools, Adobe Photoshop is utilized as the primary software for 2D graphics processing and user interface (UI) design. It facilitates the creation and refinement of essential UI elements, such as the main menu, health bars, and icons, ensuring a polished and professional visual style. Furthermore, Photoshop is used to adjust textures and sprite assets to maintain visual consistency before integrating them into the Unity environment.



Figure Error: Reference source not found-3: Unity Asset Store

Moreover, the visual environment of the game is constructed using assets sourced from the Unity Asset Store, which provides a wide range of high-quality, pre-built models, textures, animations, and environmental components. These assets are utilized to design the game world, including terrain, architectural elements, lighting effects, and atmospheric details, allowing the development team to focus on gameplay mechanics rather than creating all visual resources from scratch. The use of Asset Store environments ensures visual consistency, accelerates development time, and enhances the overall aesthetic quality of the game. Furthermore, selected assets are customized and optimized within Unity to align with the game's artistic direction and performance requirements, contributing to an immersive and believable game environment.

2.1.3. Development tools and version control

To support a professional and efficient development workflow, the team adopts a set of industry-standard tools that cover programming, project management, and version control throughout the software development lifecycle. These tools include Microsoft Visual Studio 2022 as the primary development environment, Unity Hub for managing Unity projects and editor versions, and Unity DevOps for source code and asset version control. Together, they ensure consistency, collaboration, and maintainability during the development process.



Figure Error: Reference source not found-4: Unity Engine and Microsoft Visual Studio 2022

Microsoft Visual Studio 2022 is used as the primary Integrated Development Environment (IDE) for the project. It provides a powerful and comprehensive set of tools for writing, editing, and debugging source code, especially for C# development in Unity. Features such as intelligent code completion, built-in debugging, error detection, and project dependency management help developers identify issues early and maintain clean, well-structured code. Visual Studio also integrates seamlessly with Git-based version control systems, enabling developers to manage commits, branches, and merges directly within the IDE, thereby improving productivity and development efficiency.



Figure Error: Reference source not found-5: Unity Hub

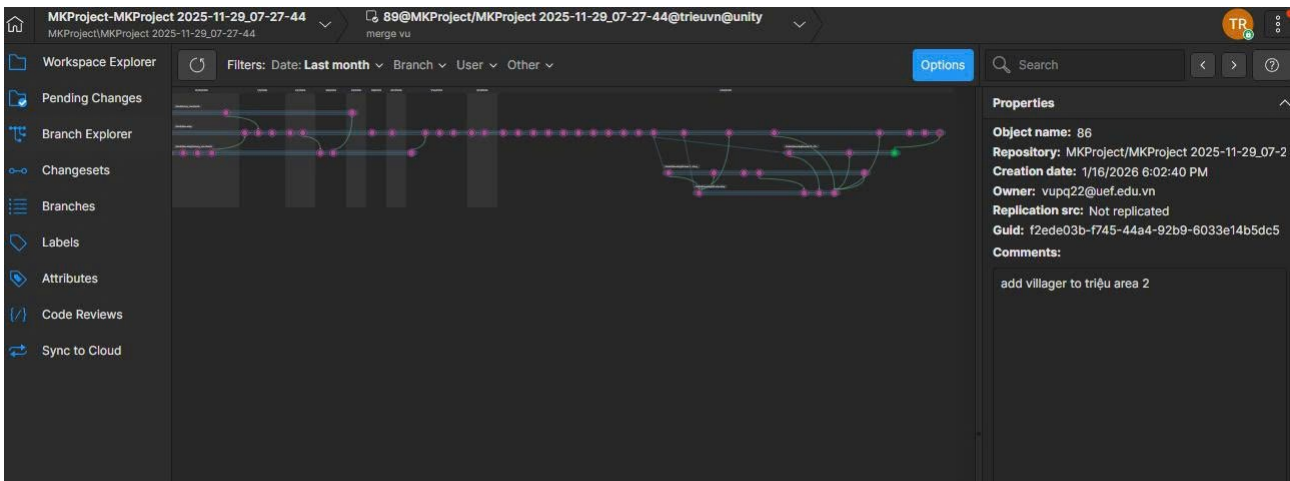
Unity Hub functions as the central management platform for all Unity-related development activities. It allows the team to install, manage, and switch between different versions of the Unity Editor, ensuring compatibility and consistency across the project lifecycle. Through Unity Hub, developers can easily create new projects, open existing ones, and manage project settings in a unified interface. This tool is particularly important for collaborative development, as it ensures that all team members work with the same Unity

version and environment, reducing configuration issues and improving overall workflow stability.



Figure Error: Reference source not found-6: Unity DevOps

Unity DevOps is utilized as the version control solution to manage source code and project assets effectively. It provides robust version control features such as change tracking, commit history, branching, and merging, which are essential for collaborative development. By using Unity DevOps, the team can safely work on different features in parallel while minimizing conflicts and data loss. Additionally, it supports large binary files commonly used in game development, such as textures, models, and audio assets. This ensures reliable asset management and maintains project integrity throughout the development process.



Unity DevOps Commit and Branch History Visualization

Moreover, Unity DevOps provides a clear commit and branch history, helping the team track changes, manage parallel development, and quickly restore stable versions when needed, thereby ensuring a stable and well-organized development workflow. issues arise. Overall, Unity DevOps plays a crucial role in ensuring efficient collaboration, stable development workflows, and long-term maintainability of the game project.

2.2. Audacity or other audio tools:

To enhance the overall quality and immersion of the game experience, audio tools are used to support sound design, music editing, and media processing during development. These tools help ensure that sound effects, background music, and multimedia elements are clear, well-balanced, and properly optimized before being integrated into the game engine. The use of dedicated audio and media tools contributes significantly to the professionalism and polish of the final product.



Figure Error: Reference source not found-7: Audacity

Audacity is utilized as an audio editing tool for creating and refining game sound assets. It supports recording, editing, and processing various audio elements such as sound effects, ambient sounds, and background music. With features including trimming, noise reduction, normalization, and audio format conversion, Audacity helps maintain consistent audio quality and appropriate volume levels. These processed audio files are prepared to meet technical requirements and ensure smooth integration into the game, improving immersion and player experience.



Figure Error: Reference source not found-8: CapCut

CapCut is used as a supporting media editing tool for handling audio-related multimedia content. It assists in editing and synchronizing audio with visual elements, such as trailers, cutscene previews, or promotional materials related to the game. CapCut provides an intuitive interface and practical features for basic audio enhancement and media composition, allowing the development team to efficiently prepare high-quality multimedia content that complements the game's presentation and documentation.

2.3. Requirement analysis

2.3.1. Functional requirements

Based on the intended gameplay flow and system design, the game is required to support a set of core functional features that ensure a coherent and playable experience. At the most fundamental level, the system must allow the player to control a character within the game world, including movement, camera handling, and interaction with the environment. Player actions such as attacking, defending, evading, and receiving damage must be processed accurately and reflected immediately through visual and audio feedback.

The game must also implement enemy behavior systems that enable non-player characters to detect the player, respond to proximity, and execute predefined actions such as patrolling, engaging, and retreating. Combat interactions between the player and enemies must correctly calculate damage, health reduction, and defeat conditions. In addition, the game must manage player states, including health and stamina values, and handle transitions such as entering combat, player defeat, or level completion.

Furthermore, the system must provide basic in-game interface elements to support gameplay clarity. These include displaying player health and stamina indicators, showing damage feedback, and presenting simple menus such as pause and restart options. Together, these functional requirements ensure that the game operates as an interactive and self-contained single-player experience.

2.3.2. Non functional requirements

In addition to gameplay features, the project must satisfy several non-functional requirements to ensure overall quality and stability. From a Performance perspective, the game is required to run smoothly in real time, maintaining consistent frame rates and responsive controls during combat and exploration scenarios.

Regarding Reliability and Stability, the game must handle state transitions correctly without crashes or unexpected behavior, particularly during combat interactions, enemy encounters, and player defeat scenarios. Core systems should function consistently across repeated gameplay sessions.

In terms of Usability, the game interface must be clear and intuitive, allowing players to easily understand their current status and available actions without extensive instructions. Visual indicators, control responsiveness, and feedback cues are designed to support player awareness and decision-making. Collectively, these non-functional requirements contribute to a stable, understandable, and enjoyable gameplay experience suitable for an academic game development project.

CHAPTER 3. SYSTEM DESIGN

3.1. Character Control & Movement

The character control system is the core component of the gameplay experience in Mountain Kin: The King's Demand. It is designed to provide responsive and fluid movement, allowing players to navigate the environment and engage in combat effectively. The system is built using Unity's CharacterController component, which handles collision and slope limits without the unpredictability of raw physics forces.

3.1.1. Main Character Assets

The main character is designed as a skilled warrior, equipped with light armor and a melee weapon to fit the game's action-adventure theme. The 3D model is rigged with a standard humanoid skeleton, ensuring compatibility with Unity's Avatar system and Mecanim animation engine. This setup allows for smooth transitions between various states such as Idle, Run, Attack, and Jump.

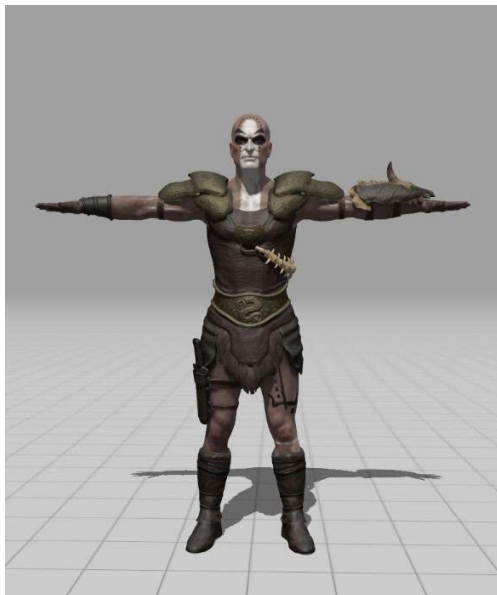


Figure Error: Reference source not found-9: The main character model used in the game

3.1.2. Enemy Assets

To complement the main character, the game features distinct enemy types designed to challenge the player's combat skills. The primary enemy model is designed as a heavily armored knight, aligning with the game's medieval combat theme. These enemies serve as the main obstacles in the level, requiring the player to utilize timing and strategy to defeat them.



Figure Error: Reference source not found-10: Enemy character model (Armored Knight)

Regarding AI Pathfinding, the enemy movement is driven by Unity's NavMeshAgent system to ensure intelligent navigation. This allows the enemies to effectively navigate the game world, automatically avoiding static obstacles and calculating the shortest path to chase the player immediately when detected.

In terms of State Management, similar to the player, enemies operate on a Finite State Machine (FSM) logic. They dynamically switch between distinct behaviors such as Patrol, which involves moving between waypoints, Chase, where the enemy runs towards the player, and Attack, executing damage logic when the target is in range.

3.1.3. Boss Assets

The Boss character represents the ultimate challenge in Mountain Kin: The King's Demand. Visually distinct from standard enemies, the Boss features a larger, more muscular build and wields a unique weapon to signify a higher threat level. This character is designed to test the player's mastery of the game's combat mechanics, requiring precise timing and dodging skills to defeat.

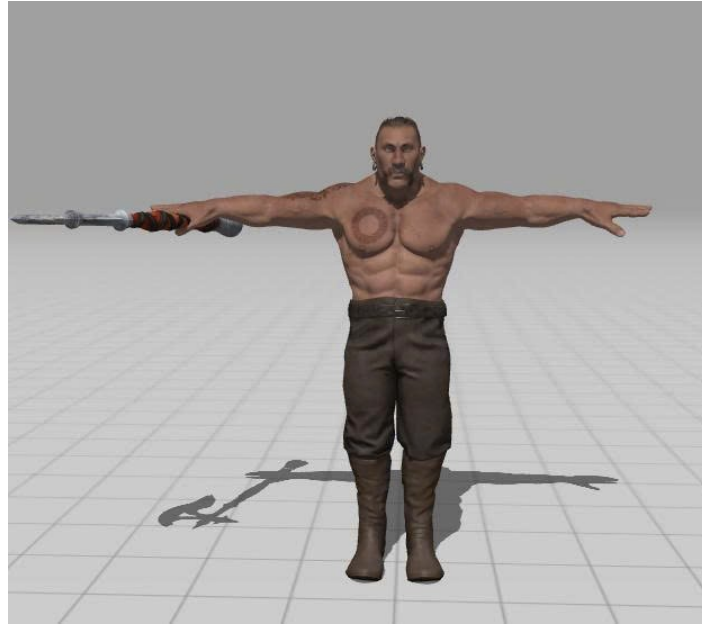


Figure Error: Reference source not found-11: Boss character model

In terms of Boss Mechanics, the entity possesses enhanced attributes compared to regular enemies, characterized by significantly higher Health Points (HP) and the capacity to deal massive damage per hit. The Boss AI is designed with specific attack patterns featuring slower but heavier animations, which grants the player a crucial window to dodge or parry before executing a counterattack.

Regarding difficulty progression, the system includes an optional Phasing mechanic to elevate the challenge. When the Boss's health falls below 50%, it enters an "Enraged" state, dynamically increasing attack speed and aggression to heighten the overall difficulty curve for the player.

3.1.4. Player System

The Player System acts as the central logic unit that governs the main character's behavior, bridging the gap between user input and visual representation. This system is implemented using Unity's Mecanim Animation System, which organizes character actions into a comprehensive Finite State Machine (FSM).

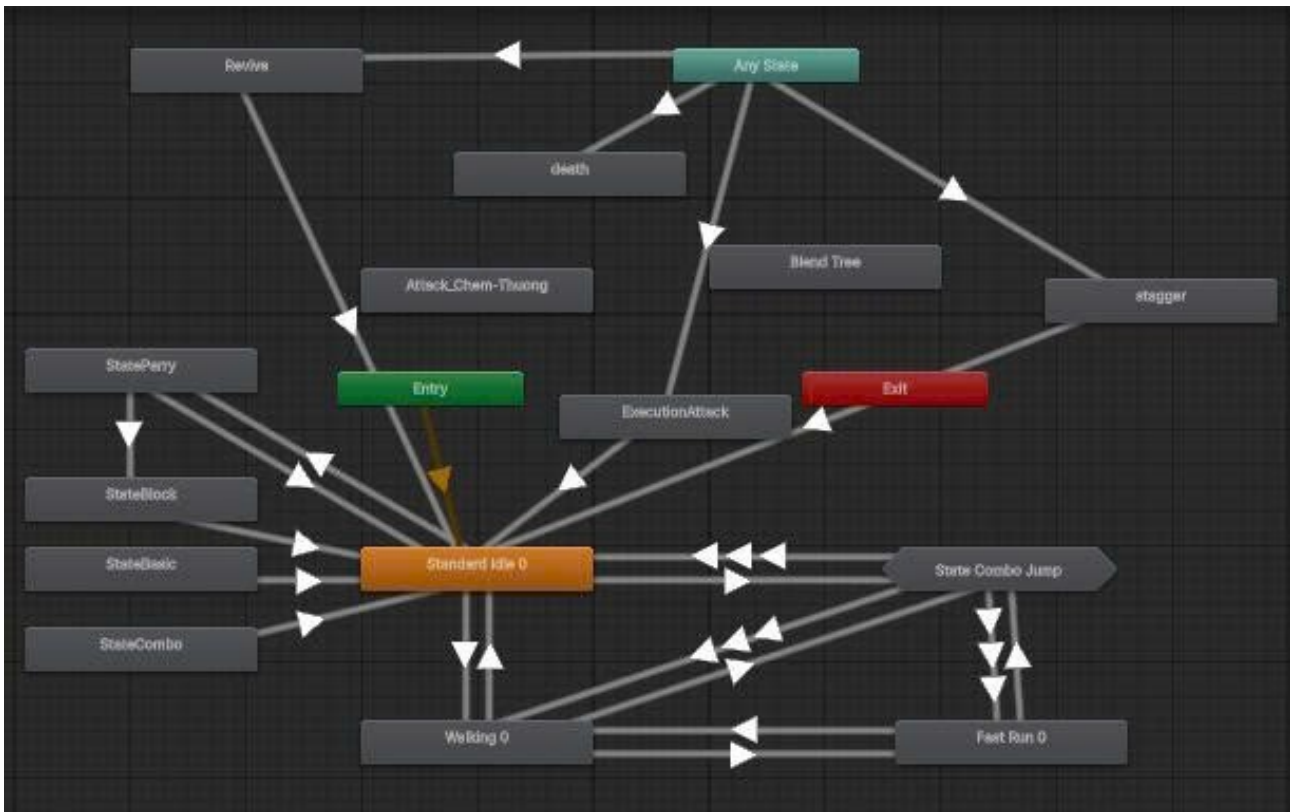


Figure Error: Reference source not found-12: Player Animator Controller State Machine

As illustrated in Figure 3.4, the character's behavior is structured as a network of interconnected states. Within the Locomotion Layer, the "Standard Idle" node serves as the default entry point. Based on input parameters such as speed variables, the system transitions seamlessly into "Walking" or "Fast Run" states, utilizing Blend Trees to ensure smooth acceleration throughout the movement.

Regarding Combat States, the system handles complex combat logic through specific trigger parameters. The "Attack" and "Combo" nodes allow the player to chain strikes together, while defensive states like "Block" and "Parry" provide tactical options during fights. Furthermore, in terms of Reactionary States to ensure gameplay feedback, the "Hit" state is triggered immediately upon taking damage, effectively interrupting other actions, with the "Death" state serving as the terminal node activated when the character's health reaches zero.

3.1.5. Player Component Configuration

To ensure modularity and ease of tuning, the player's logic is decoupled into separate components managed via the Unity Inspector. As shown in Figure 3.5, the configuration utilizes a data-driven approach to handle complex combat mechanics.

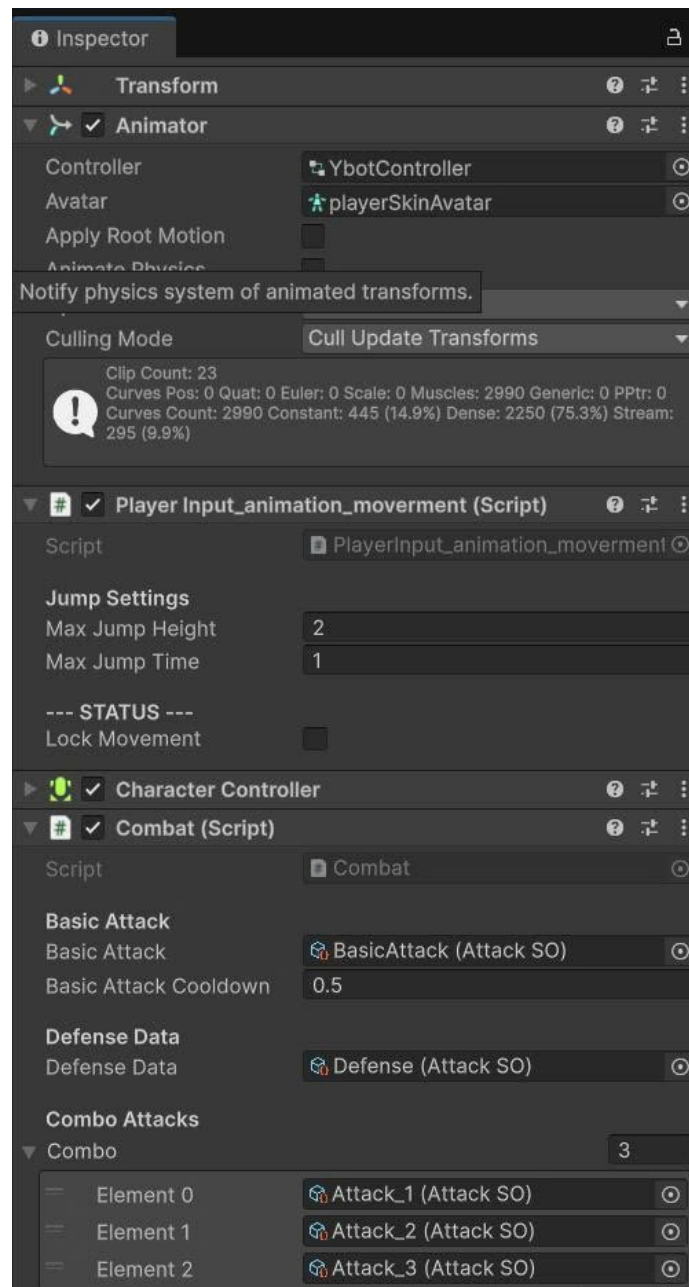


Figure Error: Reference source not found-13: Player Inspector configuration showing Movement and Combat scripts

In terms of Component Analysis, the Movement Script, specifically identified as `PlayerInput_animation_movement`, is responsible for handling physics calculations. Key parameters such as `Max Jump Height` and `Max Jump Time` are exposed publicly in the inspector, effectively allowing designers to fine-tune the gameplay feel without the need to modify the underlying code.

Regarding the Combat Script and Scriptable Objects, the system manages attack logic through a flexible, modular approach. Instead of hard-coding damage values, the architecture utilizes Scriptable Objects, referenced as "Attack SO" in the Inspector. This reflects a Data-

Driven Design where each attack, such as BasicAttack, Attack_1, or Attack_2, is created as a separate data asset containing its unique properties including damage, cooldown, and animation clips.

Furthermore, within the Combo System, the Combo Attacks list allows developers to easily sequence these assets. This structure enables the creation of complex combo chains by simply dragging and dropping new Attack Objects into the array, thereby streamlining the process of defining combat sequences.

3.1.6. Combat Feedback & Audio Integration

To create an immersive combat experience, the system relies on immediate audio-visual feedback and state management. Figure 3.6 demonstrates the configuration of these feedback loops in the Unity Inspector.

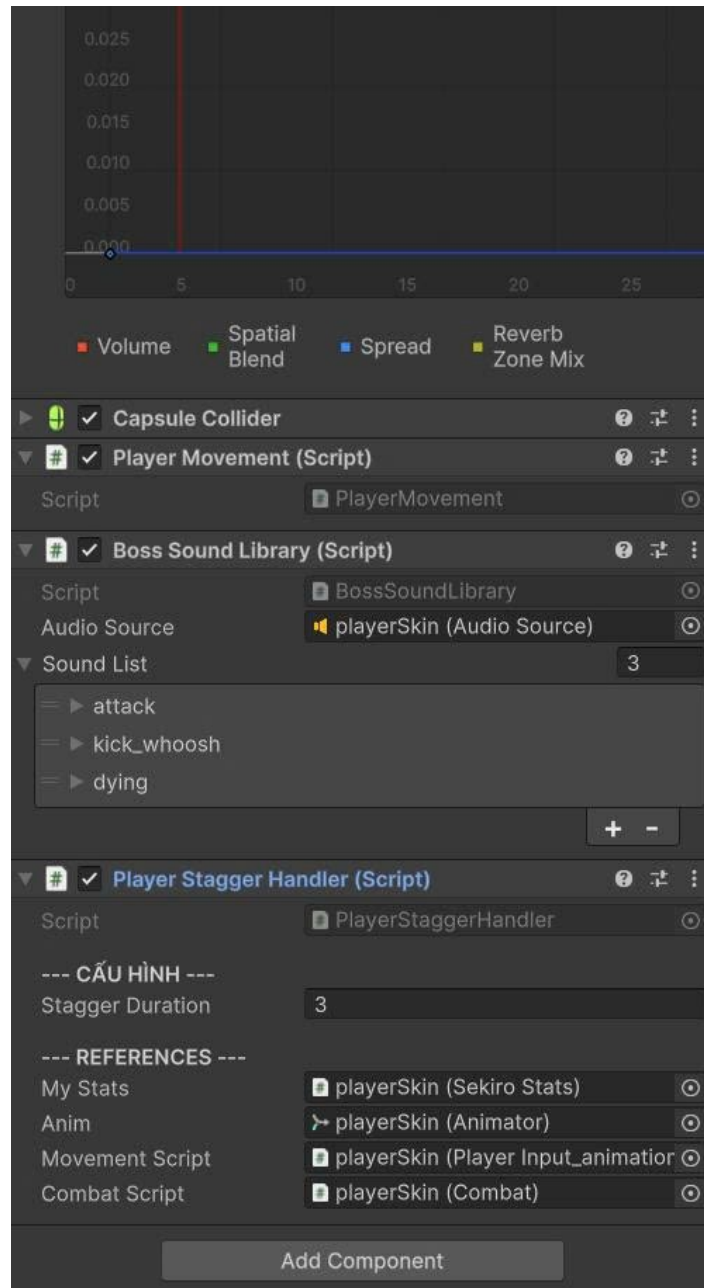


Figure Error: Reference source not found-14: Audio Library and Stagger Handler configuration

Regarding Feedback Mechanisms, specifically Audio Management, the Boss Sound Library script functions as a centralized audio manager for the character. It maintains a comprehensive list of AudioClip references, such as Attack, Kick Whoosh, and Dying, and plays them dynamically via the attached Audio Source. This configuration allows sound effects to be triggered precisely by Animation Events, ensuring perfect synchronization with visual actions.

Furthermore, the system incorporates a Stagger Mechanic through the Player Stagger Handler, which implements the "Poise" or "Stun" mechanic often found in hardcore action

games. In terms of functionality, when the character receives heavy damage, this script is triggered to immediately transition the entity into a "Staggered" state. Regarding configuration, as displayed in the Inspector, developers have the ability to adjust the Stagger Duration, for instance setting it to 3 seconds, which determines how long the character remains vulnerable. The script actively communicates with the Movement and Combat scripts to temporarily disable input, thereby forcing the player to adopt a defensive strategy or suffer the consequences.

3.1.7. Survival & Recovery Mechanics

To balance the difficulty of combat, the player is equipped with survival tools that allow for health recovery and a "second chance" mechanic. Figure 3.7 illustrates the configuration of these systems in the Inspector.



Figure Error: Reference source not found-15: Healing and Revival configuration settings

In terms of System Analysis, specifically regarding Consumable Healing, the script implements a limited-use healing item system comparable to the "Estus Flask" mechanics found in Soulslike games. Developers have the ability to configure specific parameters such as the Max Heal Charges, currently set to 3, and the Heal Amount of 40 HP per use. This inherent limitation is designed to force players to utilize healing resources strategically rather than relying on spamming the ability. Additionally, regarding user feedback, upon use, the system immediately triggers a Heal VFX Prefab and updates the UI text to display the remaining charges.

Furthermore, the Revival Mechanic introduces a forgiveness system through the "Revival Settings" configuration. The functionality is governed by the "Is Revivable" boolean, which allows the player to resurrect exactly once per life or checkpoint. In terms of game balance, the Revive HP Percent, set to 0.8 or 80%, determines the specific amount of health the player recovers upon resurrection, ensuring they return to the fight with a fighting chance but remain not fully invulnerable.

3.1.8. Character Stats & Posture System

Inspired by the mechanics of modern action games (specifically the "Sekiro" style), the character's survival logic goes beyond a simple Health bar. The system implements a dual-meter system managing both Health Points (HP) and Posture. Figure 3.8 details the configuration of this advanced attribute system.

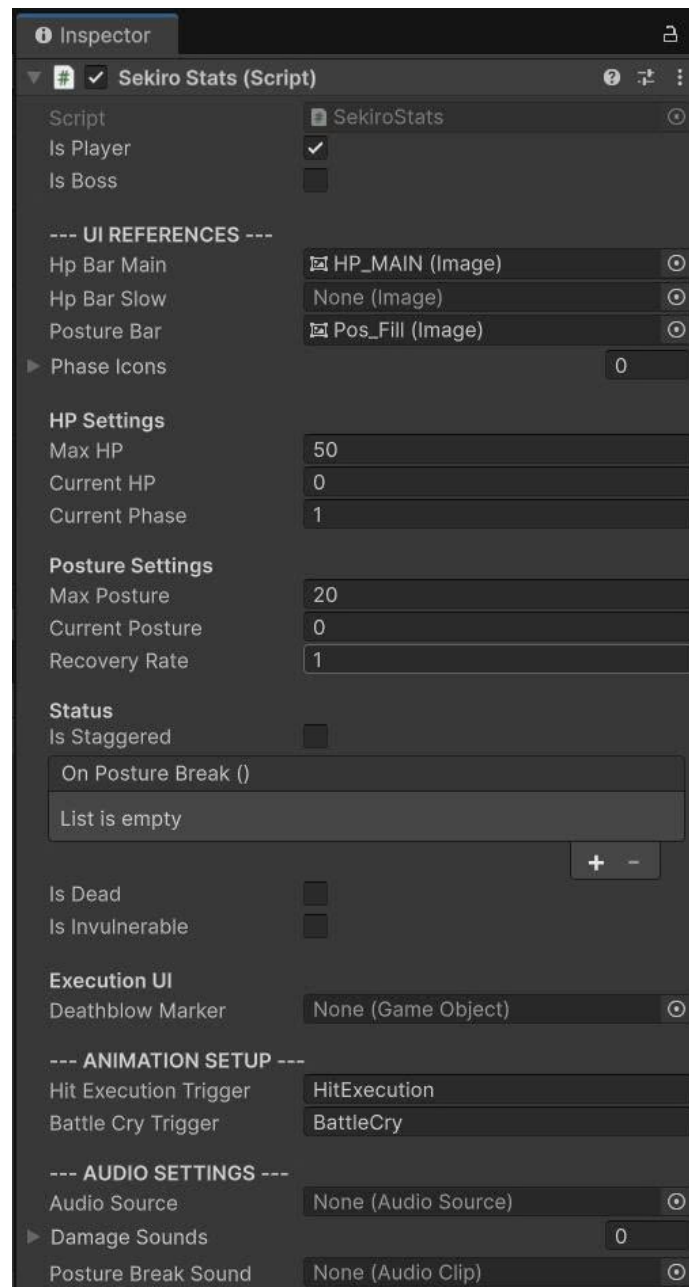


Figure Error: Reference source not found-16: Character Stats and Posture configuration (SekiroStats)

In terms of System Mechanics, the architecture employs a Dual-Attribute System consisting of standard Health (HP), where depletion results in character death, and a dynamic Posture gauge. This gauge accumulates as the character blocks or sustains hits. Once the Current Posture reaches the Max Posture limit, configured at 20 in the Inspector, the character enters a "Posture Break" state, rendering them vulnerable to critical attacks. From a technical perspective, the script integrates directly with UI elements such as the Hp Bar Main and Posture Bar. This implementation utilizes the Observer pattern to ensure that stat changes are

immediately reflected on-screen without the need for per-frame checking, thereby optimizing overall performance.

Furthermore, regarding Execution and Deathblows, the system supports a finishing move mechanic facilitated by the Deathblow Marker. When an enemy's posture is comprehensively broken, a visual indicator appears, granting the player the opportunity to perform a one-hit kill or inflict massive damage. To ensure visual fidelity, the script coordinates closely with the Animator via specific triggers, such as the HitExecutionTrigger, ensuring that the cinematic execution animation plays correctly and synchronizes perfectly whenever a deathblow is successfully performed.

3.1.9. Advanced Combat Mechanics & Target Lock System

To elevate the combat from simple hack-and-slash to a tactical duel experience, the system incorporates advanced targeting and sensory feedback mechanisms. Figure 3.9 displays the Inspector setup for these critical subsystems.

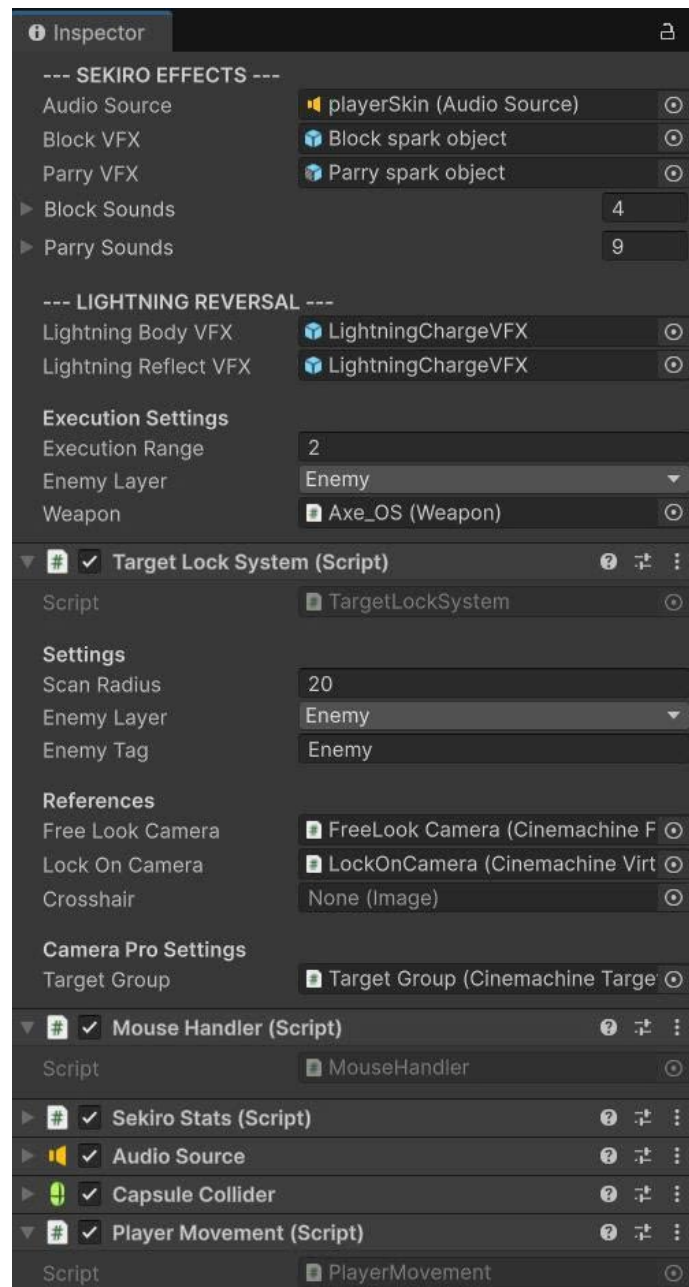


Figure Error: Reference source not found-17: Target Lock System and Combat Effects configuration

In terms of System Components, specifically the Target Lock System, the script integrates seamlessly with Cinemachine to manage camera behavior. It switches dynamically between a Free Look Camera for exploration and a Lock On Camera for focused combat, ensuring the player always faces their opponent during duels. Furthermore, the detection logic employs a Scan Radius, set to 20 units, to identify objects on the "Enemy" layer; when a target is locked, a Crosshair UI element is enabled to provide visual confirmation.

Regarding Sekiro-Style Effects such as Parry and Block, the system prioritizes feedback to reward precise timing. Visually, specific particle effects—Block VFX and Parry VFX—

are spawned at the point of impact; a standard block produces a minor spark, whereas a perfect parry generates a larger, brighter flash. Simultaneously, audio feedback is managed through separate arrays for Block Sounds and Parry Sounds, allowing for varied impact noises that prevent audio repetition and enhance the visceral nature of combat.

Additionally, the Inspector reveals a specialized Special Mechanics section featuring a "Lightning Reversal" system. This component handles distinct visual states, differentiating between Lightning Body VFX and Reflect VFX, to facilitate a high-level counter-attack move that allows the player to effectively catch and redirect incoming elemental attacks.

3.1.10. Combat Data Architecture

To efficiently manage the complexity of combat moves and combos, the project adopts a Data-Driven Architecture using Unity's Scriptable Objects. As detailed in the Inspector view (Figure 3.10), the combat logic is separated from the combat data.

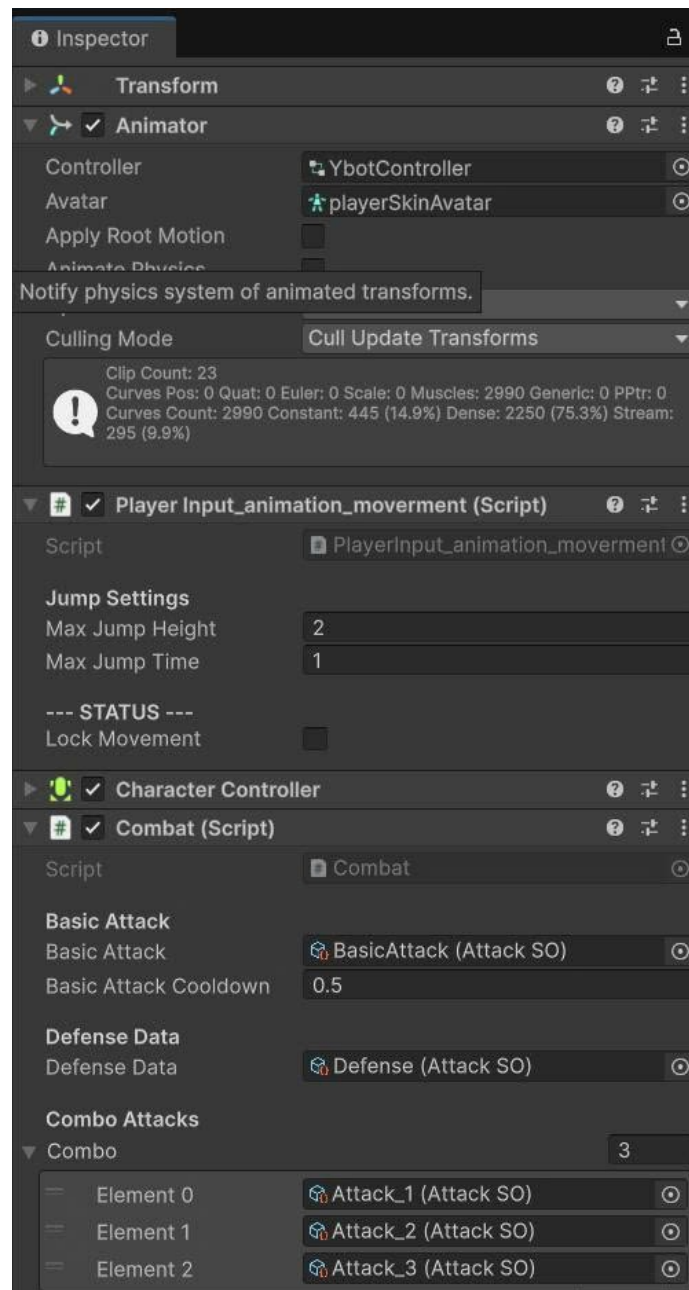


Figure Error: Reference source not found-18: Combat Script utilizing Scriptable Objects for attack data

Regarding Technical Implementation, the system utilizes Attack Scriptable Objects (SO) to effectively decouple data from logic. Instead of hard-coding attack parameters directly inside the script, each specific move, such as Basic Attack, Attack_1, or Attack_2, is created as an independent Asset file within the project folder. These assets encapsulate critical operational data, including the specific Animation Clip to be played, the Damage Value determining the amount of health to subtract, and the Cooldown period required before the attack can be utilized again.

Furthermore, the architecture supports a Modular Combo System wherein the Combo Attacks list in the Inspector functions as a dynamic array. This design empowers developers to easily construct and modify combo chains by simply dragging and dropping these Attack SOs into the designated list elements. Consequently, this architecture facilitates rapid iteration, effectively allowing game designers to tweak damage figures or rearrange combo sequences directly in the editor without the need to modify the source code, thereby significantly reducing the risk of introducing programming bugs.

3.1.11. Input Mapping Configuration

The game uses a hybrid Keyboard + Mouse control scheme optimized for PC platform:



Figure Error: Reference source not found-19: PC-Based Keyboard and Mouse Input Configuration

Technical Implementation: Player input is handled using Unity's New Input System, which ensures reliable and frame-accurate input detection. This system supports cross-platform compatibility, enabling future integration with additional input devices such as game controllers. Furthermore, input processing is decoupled from core gameplay logic, improving system modularity and maintainability.

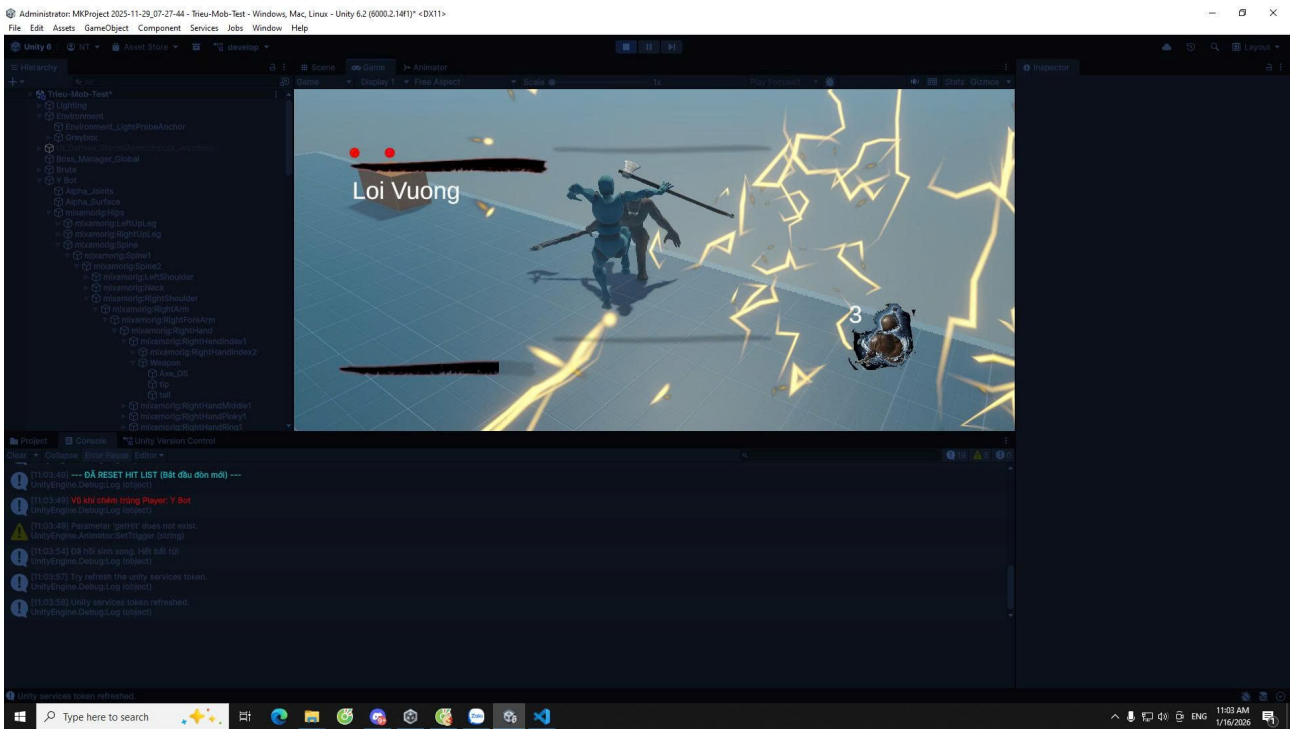


Figure Error: Reference source not found-21: Collision-Based Defensive Interaction

The jumping and aerial movement mechanics are integrated into the combat system to enhance gameplay depth and flexibility. The player is able to transition smoothly between ground-based movement and airborne actions, including aerial attacks. These mechanics allow players to avoid enemy attacks, control vertical space, and execute offensive maneuvers strategically. Combat feedback, delivered through visual effects and damage indicators, reinforces player actions and supports informed decision-making during fast-paced encounters. This design encourages dynamic movement and rewards skillful execution in combat scenarios.

3.2. Collision Detection & Interaction

The defensive counter mechanism is implemented as a collision-based interaction between the player's guard action and incoming enemy attacks. When an enemy strike intersects with the player's defensive collision boundary at the correct timing window, the system registers a successful defensive interaction. This interaction prevents damage application and alters the enemy's state, creating a brief vulnerability.

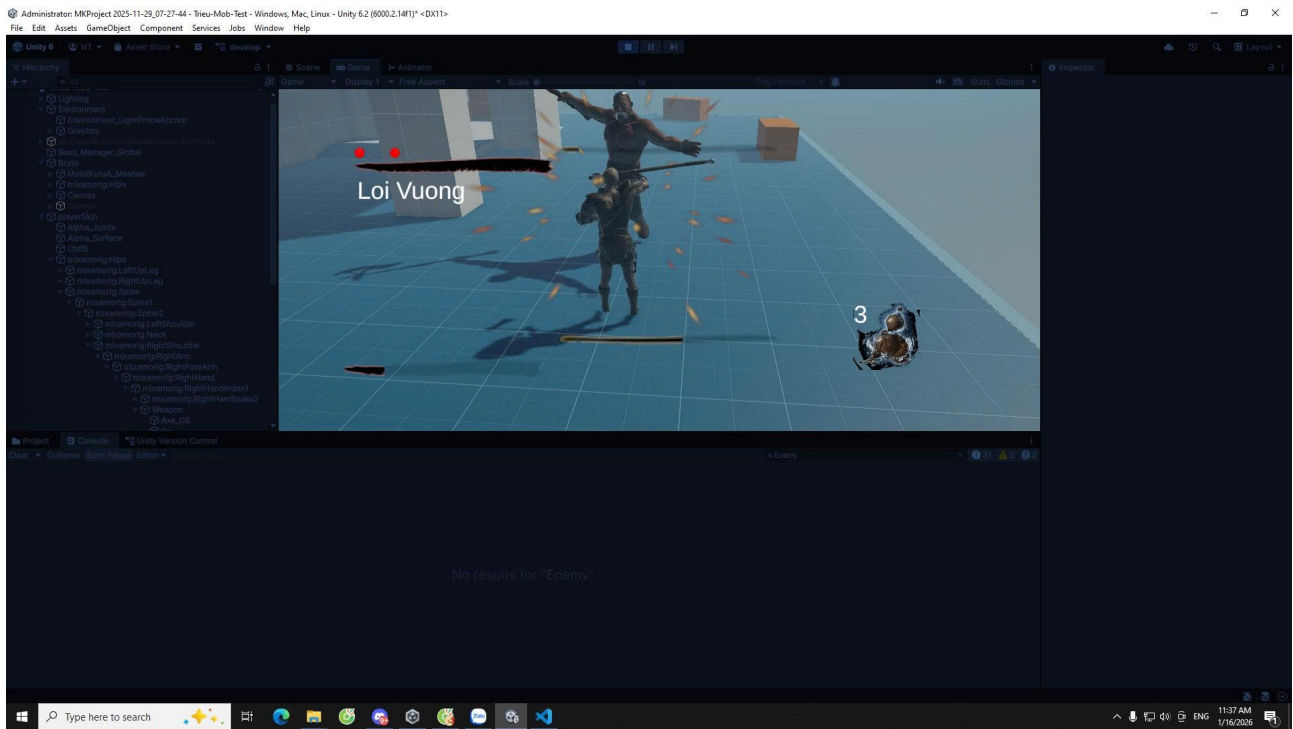


Figure Error: Reference source not found-22: Parry System Mechanics

By relying on precise collision timing rather than continuous blocking, the system ensures that defensive interactions are intentional and skill-dependent. This approach integrates defensive mechanics into the collision detection framework, allowing combat outcomes to be determined by accurate interaction handling instead of passive defense. As a result, collision-based interactions contribute directly to realistic and responsive combat behavior.

3.3. Score System Design

The score system is designed as a progression-based mechanism that evaluates player performance at the moment the player successfully deals decisive damage to the target. Rather than continuously accumulating points throughout gameplay, the score is calculated when the player completes the main combat objective of the level. This design ensures that progression is directly tied to combat success and objective completion.

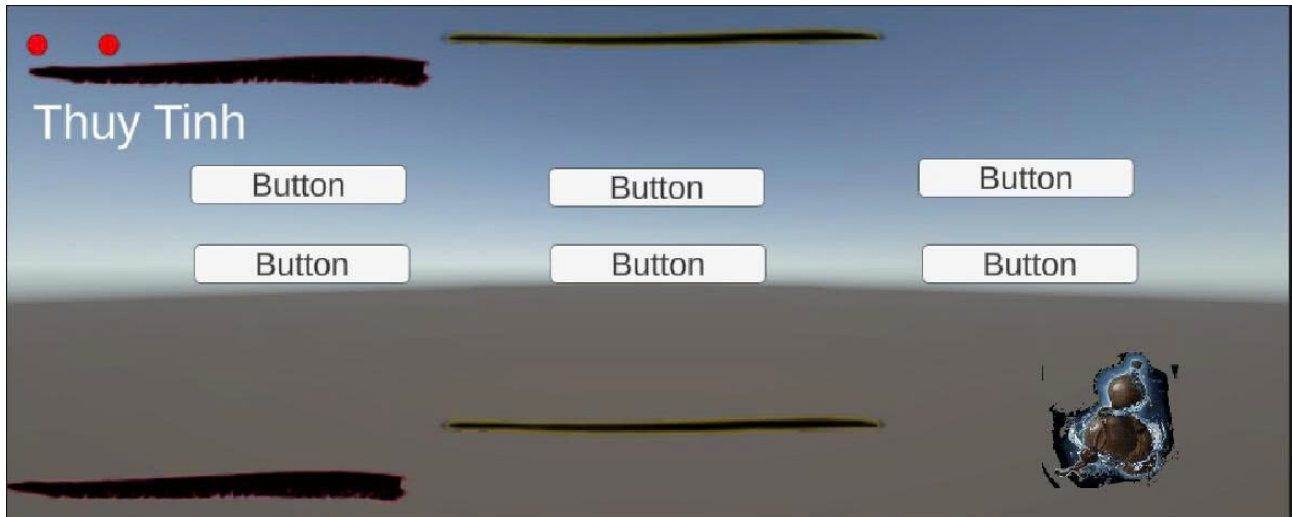


Figure Error: Reference source not found-23: Combat Result and Score Evaluation Interface

Score calculation is triggered when the player character inflicts damage on the target that results in defeating the enemy or fulfilling the level's combat requirement. At this moment, the system evaluates relevant gameplay parameters, such as the player's remaining health and combat state, to generate the final score for the stage. This approach emphasizes efficiency and survival, rewarding players who complete encounters while maintaining control and minimizing damage taken.

Once the score is calculated, it functions as a condition for level completion. A successful score evaluation confirms that the player has met the requirements to advance, allowing the game to transition smoothly to the next stage of progression. Upon completion, the gameplay state shifts from active combat to narrative delivery, triggering a story-related video sequence that continues the game's storyline.

From a user interface perspective, the score result is presented clearly to inform the player of their performance before transitioning to the narrative content. This structured flow—from combat, to score evaluation, to story progression—reinforces the connection between

gameplay mechanics and narrative development. Overall, the score system acts not only as a performance indicator but also as a key progression trigger that links combat success with story advancement.

3.4. User Interface (UI) Design

The user interface is designed to support gameplay flow, clarity, and player immersion while maintaining a consistent visual identity throughout the game. UI elements are structured to present essential information clearly and efficiently, ensuring that players can focus on gameplay without unnecessary distraction. The overall design emphasizes simplicity, readability, and consistency across different game states.



Figure Error: Reference source not found-24: Unblockable Mechanic: Lightning Counter Tutorial

The lightning-based attack is designed as an unblockable mechanic that cannot be parried or defended using standard defensive actions. Players are required to respond by performing a jump to avoid direct damage and then executing a counteraction to release the lightning energy safely. This mechanic forces players to quickly recognize specific attack patterns and react with precise timing rather than relying on defensive habits. By introducing attacks that demand unique responses, the system increases combat variety and encourages adaptive decision-making during encounters.

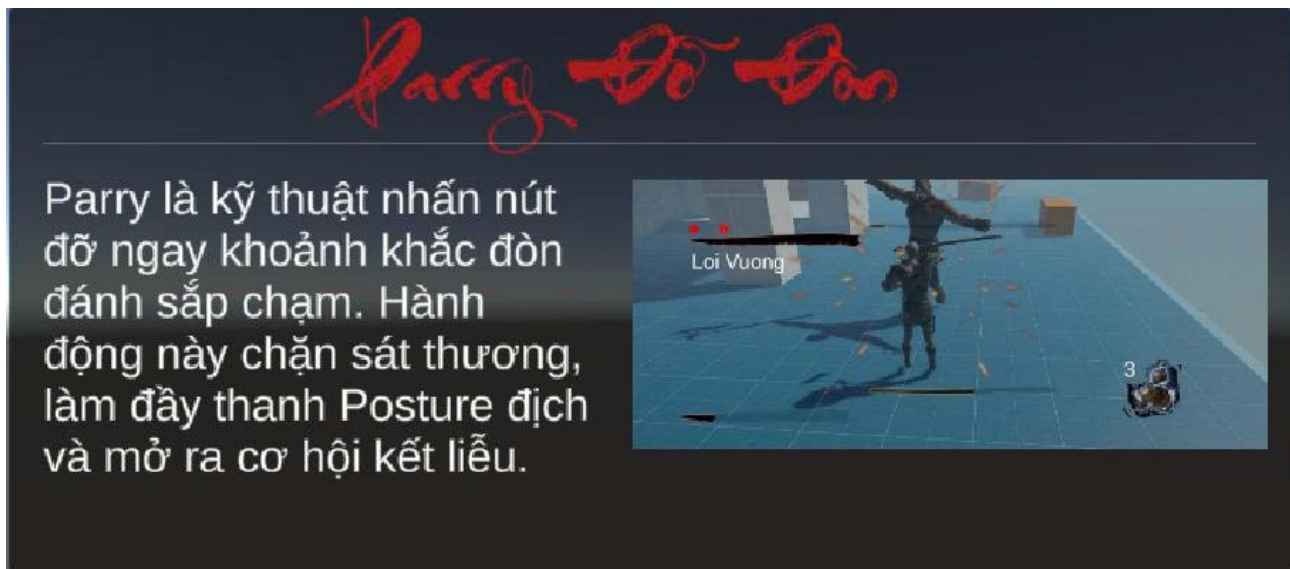


Figure Error: Reference source not found-25: Parry Mechanic Tutorial Interface

The parry system is a high-risk, high-reward defensive mechanic that allows players to negate incoming damage by pressing the defensive input at the exact moment an enemy attack lands. A successful parry prevents damage, reduces the enemy's posture or balance, and creates an opening for counterattacks. This system rewards precise timing and player skill, reinforcing an aggressive yet calculated combat style. By integrating parry mechanics into combat, the game emphasizes mastery and situational awareness as key factors for success.



Figure Error: Reference source not found-26: Menu Interface Design

The main menu serves as the primary navigation interface and provides players with clear options to start the game, access settings, or exit the application. Menu elements are logically arranged and visually distinct, allowing users to understand available actions immediately.

This structured layout ensures ease of use and creates a smooth transition from the menu into gameplay.

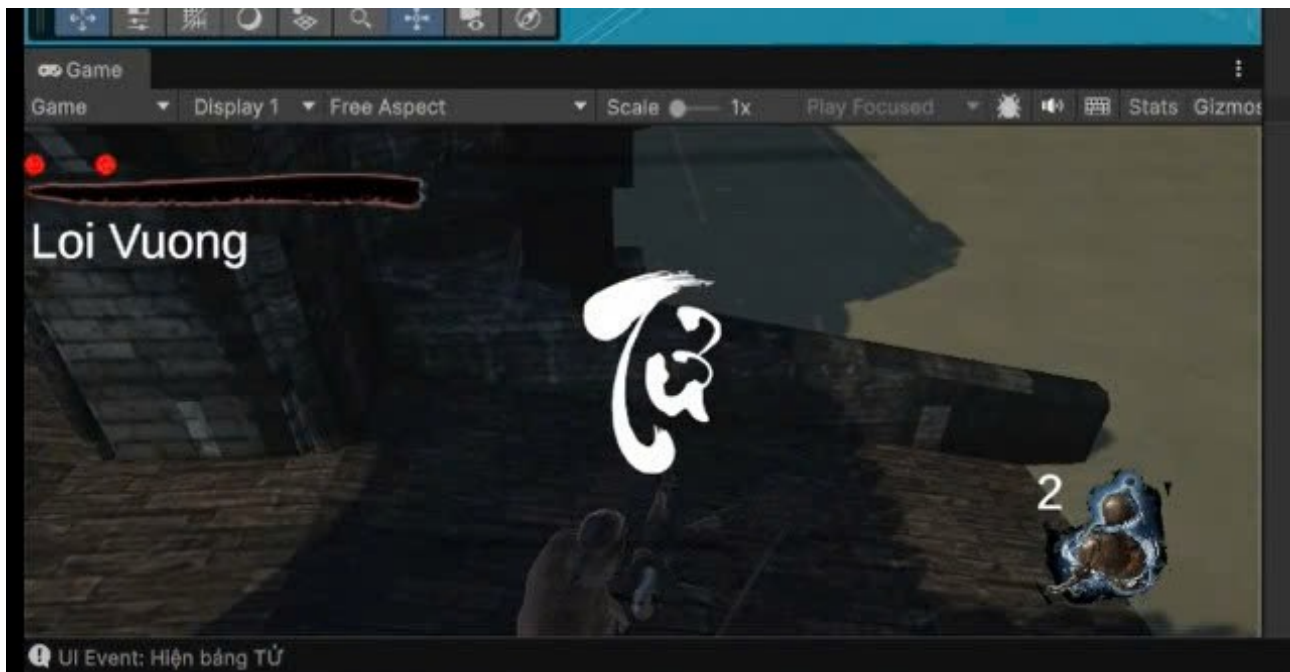


Figure Error: Reference source not found-27: Game Over Screen Design

The game over screen is designed to clearly indicate the end of a gameplay session and communicate the player's final performance. It presents essential information such as the game state outcome and relevant options, including restarting the game or returning to the main menu. By overlaying the gameplay scene, the screen maintains visual continuity while clearly signaling that player control has ended. This design ensures clarity and helps players transition smoothly after a failed attempt.



Figure Error: Reference source not found-28: Main Menu Interface Design

The pause screen allows players to temporarily suspend gameplay while maintaining context within the game world. When activated, it overlays the current scene and provides options such as resuming gameplay or returning to the main menu. The interface is structured to be simple and immediately understandable, enabling players to quickly manage interruptions without confusion. This approach supports usability and ensures a smooth transition back into active gameplay.

3.5. Game Progress Storage

The game progress storage system is designed to preserve the player's progress by saving both the player's score and the current game state, allowing gameplay to be resumed at a later time. Instead of restarting from the beginning, players can continue from their last recorded state, improving usability and overall player experience.

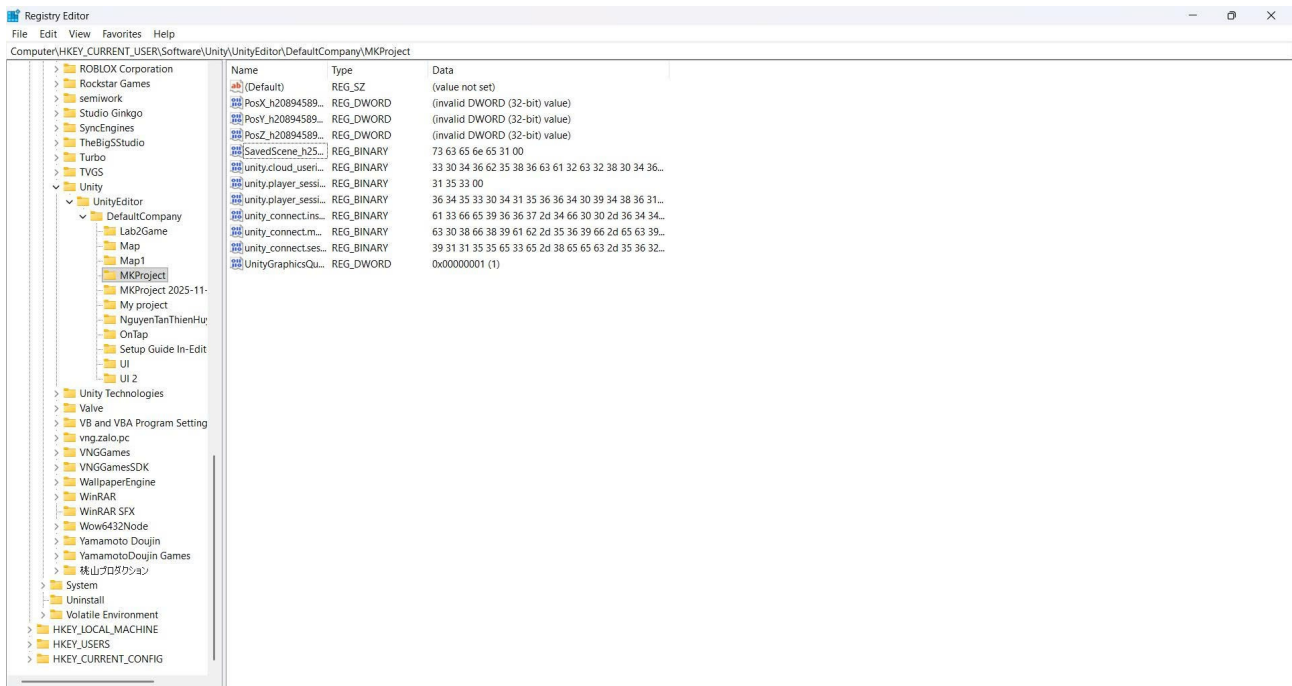


Figure Error: Reference source not found-29: Registry-Based Game Progress Storage

Player position data is stored using three-dimensional coordinates (x, y, z), which represent the exact location of the character within the game world. By recording these spatial values, the system ensures that the player character can be restored to the correct position when the game is reloaded. This approach supports seamless continuation of gameplay, particularly in complex environments or during multi-stage encounters.

In addition to positional data, the system stores relevant game state information, including the player's current score and essential asset-related states. These may include the status of enemies, progression flags, or level completion indicators that determine the current stage of the game. Storing asset-related states ensures consistency between gameplay sessions and prevents loss of progression.

All stored data is managed persistently to maintain integrity across sessions. When the game is launched again, the saved position, score, and game state are retrieved and applied to the game environment, allowing the player to resume gameplay without disrupting progression. Overall, the game progress storage system provides reliability, continuity, and support for long-term engagement.

CHAPTER 4. IMPLEMENTATION AND RESULTS

4.1. Development environment and tools

This project was developed and tested using the following hardware and software configurations. Establishing a consistent development environment is essential to ensure reproducibility, stability, and reliable performance of the game across different systems.

1. **Operating System:** Windows 10 / Windows 11 (64-bit)
2. **Game Engine:** Unity 2022.3 LTS (Long Term Support) or later
3. **Integrated Development Environment (IDE):** Microsoft Visual Studio 2022, with the *Game development with Unity* workload installed
4. **Version Control:** Unity DevOps
5. **Graphics Resources:** Unity Asset Store, used for user interface elements and texture adjustments
6. **Audio Tool:** Audacity, used for sound effect editing and processing

4.2. Step-by-Step installation guide

In order to ensure stable and flexible player input handling, the project is configured to support both the legacy Input Manager and the new Unity Input System. This approach allows the game to utilize traditional keyboard and mouse input methods while remaining compatible with modern input handling features provided by Unity.

The configuration process begins by opening the project through Unity Hub using Unity 2022.3 LTS. After the project has fully loaded, navigate to Edit → Project Settings → Player. Within the Other Settings section, locate the Active Input Handling option. This setting controls how Unity processes input events across the entire project.

By default, Unity may use either the old or the new input system. For this project, the option is explicitly set to Both, enabling simultaneous support for both systems. Once this change is applied, Unity will request a restart of the Editor. Restarting the Editor is a required step to ensure that the new input configuration is fully initialized and applied consistently across all scenes and scripts.

After restarting Unity, development continues in Microsoft Visual Studio 2022, which is integrated with Unity through the Game development with Unity workload. Visual Studio serves as the primary environment for writing and maintaining C# scripts related to player control, combat actions, interaction detection, and camera movement.

With the Both input handling mode enabled, the game can reliably process keyboard and mouse input during runtime. This includes movement commands, attack execution, defensive actions such as blocking, and contextual interactions with in-game objects or enemies. The configuration also prevents input-related conflicts that may occur when combining different input APIs.

Overall, enabling both input systems ensures consistent player control behavior during development, testing, and future expansion. It also provides a robust foundation for extending input features without requiring major changes to existing gameplay logic.

4.3. Key code snippets and explanation

4.3.1. Implementing input handling and character movement control

```
public class PlayerInput_animation_movement : MonoBehaviour
{
    //PlayerInput playerInput;
    Ybot_input playerInput;
    CharacterController characterController;
    Animator animator;

    // --- THÊM BIẾN CAMERA ---
    Transform cameraTransform;

    //hash for animator parameters
    int isWalkingHash;
    int isRunningHash;

    //input variables
    Vector2 currentMovementInput;
    Vector3 currentMovement;
    Vector3 currentRunMovement;
    Vector3 appliedMovement;

    //state variables
    bool isMovementPressed;
    bool isRunPressed;
    bool isJumpPressed;
    float rotationFactorPerFrame = 15.0f; // Giảm xuống một chút để xoay mượt hơn
    float walkMultiplier = 5f;
    float runMultiplier = 10f;

    //gravity variables
    float groundedGravity = -.05f;
    float gravity = -9.81f;

    //jump variables
    bool isJumpingPressed = false;
    float initialJumpVelocity;
    [Header("Jump Settings")]
    [SerializeField] float maxJumpHeight = 5f; // Độ cao nhảy (m)
    [SerializeField] float maxJumpTime = 0.75f; // Thời gian nhảy (giây)
    bool isJumping = false;
    int isJumpingHash;
    bool isJumpingAnimate = false;
    int jumpCount = 0;
    int jumpCountHash;

    [Header("---- STATUS ----")]
    public bool lockMovement = false;
}
```

Figure 4.3.1-1: Player Movement and Animation Controller

The code presented is the PlayerInput_animation_movement script, which serves as the central controller for the player character. This script is responsible for receiving input signals

from the player, processing custom physics calculations for movement and jumping, and synchronizing these states with the Animator system. The script utilizes Unity's CharacterController component to handle physical collisions instead of a Rigidbody, allowing for more precise and controllable movement in third-person action games.

```
//Attack variables
int isAttackHash;
Dictionary<int, float> initialJumpVelocities = new Dictionary<int, float>();
Dictionary<int, float> jumpGravities = new Dictionary<int, float>();
Coroutine currentJumpResetRoutine = null;

TargetLockSystem targetLockSystem;
// Unity Message | 0 references
void Awake()
{
    playerInput = new Ybot_input();
    characterController = GetComponent<CharacterController>();
    animator = GetComponent<Animator>();

    // --- LẤY CAMERA MAIN ---
    if (Camera.main != null)
    {
        cameraTransform = Camera.main.transform;
    }
    else
    {
        Debug.LogError("Không tìm thấy Main Camera! Hãy gán Tag MainCamera cho Camera của bạn.");
    }

    targetLockSystem = GetComponent<TargetLockSystem>(); // Lấy script Lock

    isWalkingHash = Animator.StringToHash("isWalking");
    isRunningHash = Animator.StringToHash("isRunning");
    isJumpingHash = Animator.StringToHash("isJumping");
    jumpCountHash = Animator.StringToHash("jumpCount");
    isAttackHash = Animator.StringToHash("isAttack");

    playerInput.CharacterControls.Move.started += onMovementInput;
    playerInput.CharacterControls.Move.performed += onMovementInput;
    playerInput.CharacterControls.Move.canceled += onMovementInput;
    playerInput.CharacterControls.Run.started += onRun;
    playerInput.CharacterControls.Run.canceled += onRun;
    playerInput.CharacterControls.Jump.started += onJump;
    playerInput.CharacterControls.Jump.canceled += onJump;

    setupJumpVariables();
}
```

Figure 4.3.1-2: Player Input and Animation System Initialization

The Awake function serves as the initialization point for the entire system. Here, the script establishes a connection with the new input system, Ybot_input, and registers event listeners for move, run, and jump actions. Simultaneously, this function retrieves and stores a reference to the main camera to facilitate movement direction calculations and initializes hash IDs for Animator parameters to optimize performance instead of continuous string comparisons during runtime. Finally, it calls the setupJumpVariables function to establish the initial physical parameters.

```

1 reference
void setupJumpVariables()
{
    initialJumpVelocities.Clear();
    jumpGravities.Clear();

    // XÓA HOẶC COMMENT 2 DÒNG DƯỚI ĐÂY:
    // maxJumpHeight = 5f;
    // maxJumpTime = 0.75f;

    float timeToApex = maxJumpTime / 2f;
    gravity = (-2 * maxJumpHeight) / (timeToApex * timeToApex);
    initialJumpVelocity = (2 * maxJumpHeight) / timeToApex;

    // Các tính toán cho bước nhảy 2 và 3 sẽ tự động tỷ lệ theo maxJumpHeight mỗi
    float secondJumpGravity = (-2 * (maxJumpHeight + 2)) / Mathf.Pow((timeToApex * 1.25f), 2);
    float secondJumpInitialVelocity = (2 * (maxJumpHeight + 0.8f)) / (timeToApex * 1.25f);
    float thirdJumpGravity = (-2 * (maxJumpHeight + 4)) / Mathf.Pow((timeToApex * 1.5f), 2);
    float thirdJumpInitialVelocity = (2 * (maxJumpHeight + 4)) / (timeToApex * 1.5f);

    initialJumpVelocities.Add(1, initialJumpVelocity);
    initialJumpVelocities.Add(2, secondJumpInitialVelocity);
    initialJumpVelocities.Add(3, thirdJumpInitialVelocity);

    jumpGravities.Add(0, gravity);
    jumpGravities.Add(1, gravity);
    jumpGravities.Add(2, secondJumpGravity);
    jumpGravities.Add(3, thirdJumpGravity);
}

2 references
void onJump(InputAction.CallbackContext context)
{
    isJumpPressed = context.ReadValueAsButton();
}

2 references
void onRun(InputAction.CallbackContext context)
{
    isRunPressed = context.ReadValueAsButton();
}

3 references
void onMovementInput(InputAction.CallbackContext context)
{
    // Chỉ lưu giá trị input thô (Raw Input), không tính toán vận tốc ở đây nữa
    currentMovementInput = context.ReadValue<Vector2>();
    isMovementPressed = currentMovementInput.x != 0 || currentMovementInput.y != 0;
}

```

Figure 4.3.1-3: Triple Jump Physics Calculation and Event-Based Input Handling

The `setupJumpVariables` function is responsible for calculating physical parameters for the Triple Jump mechanism. Instead of assigning a fixed jump force, this function utilizes physics formulas to calculate gravity and initial velocity based on the maximum jump height and the desired time to reach the peak. These parameters are stored in a Dictionary to apply different gravity levels and jump forces for the first, second, and third jumps, creating a sense of fluidity and diversity for the character's movement.

The input event handling functions include `onMovementInput`, `onRun`, and `onJump`. These functions operate according to the event-driven mechanism of the new Input System. Instead of checking key states in every frame, these functions are only called when the player presses or releases a key, ensuring that state variables such as `currentMovementInput`, `isRunPressed`, and `isJumpPressed` are updated accurately while optimizing performance.


```

// --- HÀM MỚI ĐỂ XỬ LÝ DI CHUYỂN THEO CAMERA ---
1 reference
void handleCameraRelativeMovement()
{
    // 1. Lấy hướng nhìn của camera
    Vector3 cameraForward = cameraTransform.forward;
    Vector3 cameraRight = cameraTransform.right;

    // 2. Loại bỏ thành phần Y để chỉ di chuyển trên mặt phẳng (xz)
    cameraForward.y = 0;
    cameraRight.y = 0;

    // 3. Chuẩn hóa vector (để độ dài luôn bằng 1)
    cameraForward.Normalize();
    cameraRight.Normalize();

    // 4. Kết hợp Input với hướng Camera
    // Input.y (W/S) đi theo Camera Forward
    // Input.x (A/D) đi theo Camera Right
    Vector3 moveDirection = (cameraForward * currentMovementInput.y + cameraRight * currentMovementInput.x);

    // 5. Gán giá trị vào biến movement (giữ nguyên Y cũ để nhảy/trọng lực hoạt động đúng)
    float currentY = currentMovement.y;
    float currentRunY = currentRunMovement.y;

    currentMovement = moveDirection * walkMultiplier;
    currentMovement.y = currentY; // Khôi phục Y

    currentRunMovement = moveDirection * runMultiplier;
    currentRunMovement.y = currentRunY; // Khôi phục Y
}

```

Figure 4.3.1-4: Camera-Based Movement Direction Processing

The `handleCameraRelativeMovement` function is the core logic for processing character movement direction. This function retrieves the forward vector of the main camera and removes the y-axis component to create an accurate movement plane. It then combines the camera's orientation with the player's input signals to ensure that when the player presses forward, the character always moves in the direction the camera is facing, regardless of which way the character is currently turned. The final result is multiplied by the walking or running speed to generate the applied movement vector.

```

1 reference
void handleJump()
{
    if (!isJumping && isJumpPressed && characterController.isGrounded)
    {
        if (jumpCount < 3 && currentJumpResetRoutine != null)
        {
            StopCoroutine(currentJumpResetRoutine);
            currentJumpResetRoutine = null;
        }
        animator.SetBool(isJumpingHash, true);
        isJumpingAnimate = true;
        isJumping = true;
        jumpCount += 1;
        jumpCount = Mathf.Clamp(jumpCount, 1, 3);
        animator.SetInteger(jumpCountHash, jumpCount);

        // Cập nhật Y cho cả 2 vector để tránh lỗi khi chuyển trạng thái
        float jumpVel = initialJumpVelocities[jumpCount];
        currentMovement.y = jumpVel;
        currentRunMovement.y = jumpVel;
        appliedMovement.y = jumpVel;
    }
    else if (!isJumpPressed && isJumping && characterController.isGrounded)
    {
        isJumping = false;
    }
}

1 reference
IEnumerator jumpResetRoutine()
{
    yield return new WaitForSeconds(0.5f);
    jumpCount = 0;
    animator.SetInteger(jumpCountHash, jumpCount);
    currentJumpResetRoutine = null;
}

```

Figure Error: Reference source not found-30: Boss jump function

The handleJump function manages the jump logic. It checks whether the player is pressing the jump button and if the character is currently grounded. If the conditions are met, the function triggers the jump animation, increments the jump counter, and applies the corresponding jump velocity from the pre-calculated data to the character's y-axis. This function also supports a consecutive jump mechanism of up to three times and automatically resets the jump counter after a period of inactivity via the jumpResetRoutine Coroutine.

```

1 reference
void handleRotation()
{
    // --- ƯU TIÊN 1: XỬ LÝ LOCK-ON (Nếu đang khóa mục tiêu) ---
    if (targetLockSystem != null && targetLockSystem.isLockedOn && targetLockSystem.currentTarget != null)
    {
        // Tính hướng từ mình -> kẻ địch
        Vector3 directionToEnemy = targetLockSystem.currentTarget.position - transform.position;
        directionToEnemy.y = 0f; // Giữ thẳng bằng, không ngửa mặt lên/xuống

        if (directionToEnemy != Vector3.zero)
        {
            Quaternion targetRotation = Quaternion.LookRotation(directionToEnemy);

            // Xoay nhân vật về phía kẻ địch
            // Dùng Slerp để xoay mượt, hoặc tăng tốc độ nếu muốn nhảy hơn
            transform.rotation = Quaternion.Slerp(transform.rotation, targetRotation, rotationFactorPerFrame * Time.deltaTime);
        }

        // QUAN TRỌNG: Return luôn để không chạy logic xoay thông thường ở dưới
        return;
    }

    // --- ƯU TIÊN 2: DI CHUYỂN BÌNH THƯỜNG (Code cũ của bạn) ---
    // (Chỉ chạy khi KHÔNG Lock-On)
    Vector3 positionToLookAt;

    // Dựa vào việc đang chạy hay đi bộ để lấy vector xoay đúng
    if (isRunPressed)
    {
        positionToLookAt.x = currentRunMovement.x;
        positionToLookAt.z = currentRunMovement.z;
    }
    else
    {
        positionToLookAt.x = currentMovement.x;
        positionToLookAt.z = currentMovement.z;
    }

    positionToLookAt.y = 0f;

    Quaternion currentRotation = transform.rotation;
}

```

Figure 4.3.1-6: Multi-Jump Handling and Jump State Management

The handleRotation function is responsible for rotating the character using two priority mechanisms. If the player is using the Target Lock feature, the character will always face the enemy to facilitate attacking and dodging. Conversely, in the free-movement state, the character rotates smoothly toward the current movement direction using the Quaternion.Slerp function, making the motion appear more natural and eliminating sudden, jerky transitions.

```

// Chỉ xoay khi có Input di chuyển
if (isMovementPressed)
{
    // Kiểm tra để tránh lỗi LookRotation(Vector3.zero)
    if (positionToLookAt != Vector3.zero)
    {
        Quaternion targetRotation = Quaternion.LookRotation(positionToLookAt);
        transform.rotation = Quaternion.Slerp(currentRotation, targetRotation, rotationFactorPerFrame * Time.deltaTime);
    }
}

1 reference
void handleAnimation()
{
    bool isWalking = animator.GetBool(isWalkingHash);
    bool isRunning = animator.GetBool(isRunningHash);

    if (isMovementPressed && !isWalking)
    {
        animator.SetBool(isWalkingHash, true);
    }
    else if (!isMovementPressed && isWalking)
    {
        animator.SetBool(isWalkingHash, false);
    }

    if ((isMovementPressed && isRunPressed) && !isRunning)
    {
        animator.SetBool(isRunningHash, true);
    }
    else if (!isMovementPressed || !isRunPressed) && isRunning)
    {
        animator.SetBool(isRunningHash, false);
    }
}

```

Figure 4.3.1-7: Player Animation State Handling

The handleAnimation function updates the parameters within the Animator Controller. Based on the current movement and running states, this function toggles boolean variables such as isWalking and isRunning. This ensures that the character's visual representation always matches its physical state, such as transitioning from idle to walking and then to sprinting.

```

1 reference
void handleGravity()
{
    bool isFalling = currentMovement.y <= 0f || !isJumpPressed;
    float fallMultiplier = 2.0f;

    if (characterController.isGrounded)
    {
        if (isJumpingAnimate)
        {
            animator.SetBool(isJumpingHash, false);
            isJumpingAnimate = false;
            currentJumpResetRoutine = StartCoroutine(jumpResetRoutine());
            if (jumpCount == 3)
            {
                jumpCount = 0;
                animator.SetInteger(jumpCountHash, jumpCount);
            }
        }

        // Reset gravity khi chạm đất
        currentMovement.y = groundedGravity;
        currentRunMovement.y = groundedGravity;
        appliedMovement.y = groundedGravity;
    }
    else if (isFalling)
    {
        float previousYVelocity = currentMovement.y;
        float newYVelocity = currentMovement.y + (jumpGravities[jumpCount] * fallMultiplier * Time.deltaTime);

        currentMovement.y = newYVelocity;
        currentRunMovement.y = newYVelocity;
        appliedMovement.y = (previousYVelocity + newYVelocity) / 2f;
    }
    else
    {
        float previousYVelocity = currentMovement.y;
        float newYVelocity = currentMovement.y + (jumpGravities[jumpCount] * Time.deltaTime);

        currentMovement.y = newYVelocity;
        currentRunMovement.y = newYVelocity;
        appliedMovement.y = (previousYVelocity + newYVelocity) / 2f;
    }
}

```

Figure 4.3.1-8: Handle gravity input

The `handleGravity` function manages custom gravity for the character. It checks whether the character is grounded or falling. If grounded, a slight downward force is applied to keep the character snapped to the terrain. If airborne, the function applies gravity that increases over time, creating a realistic free-fall trajectory. Notably, the gravity can vary depending on which jump in the sequence is being performed to create a distinct sense of weight for each movement.

```
// Update is called once per frame
@ Unity Message | 0 references
void Update()
{
    // 1. Luôn tính toán Trọng lực & Nhảy (để không bị treo lơ lửng)
    handleGravity();
    handleJump();

    // 2. Chỉ tính toán di chuyển nếu KHÔNG BỊ KHÓA
    if (!lockMovement)
    {
        // --- COPY ĐOẠN CODE CŨ CỦA BẠN VÀO ĐÂY ---
        handleCameraRelativeMovement();
        handleRotation();
        handleAnimation();

        appliedMovement.x = isRunPressed ? currentRunMovement.x : currentMovement.x;
        appliedMovement.z = isRunPressed ? currentRunMovement.z : currentMovement.z;
    }
    else
    {
        // --- NẾU BỊ KHÓA THÌ ĐỨNG YÊN ---
        appliedMovement.x = 0;
        appliedMovement.z = 0;

        // Tắt animation chạy bộ nếu đang chạy dở
        if (animator) {
            animator.SetBool("isWalking", false);
            animator.SetBool("isRunning", false);
        }
    }

    // 3. Di chuyển CharacterController (QUAN TRỌNG: Để trọng lực hoạt động)
    if (characterController.enabled)
    {
        characterController.Move(appliedMovement * Time.deltaTime);
    }
}
```

Figure 4.3.1-9: Player Controller Main Execution Loop

Finally, the `Update` function is the main loop of the script, called every frame. This function orchestrates the execution order of all the logic functions mentioned above. First, it consistently calculates gravity and jumping to ensure physics operate continuously. Then, it checks the `lockMovement` variable; if movement is not locked, it proceeds with calculating direction, rotation, and animation. Finally, it calls the `Move` method of the `CharacterController` using the synthesized movement vector to execute the character's actual displacement within the game world.

4.3.2. Boss Artificial Intelligence and Combat Logic

```

public enum BossState { Chasing, Attacking, Cooldown, PreparingLightning, PhaseTransition }
public BossState currentState;

[System.Serializable]
public class AttackConfig
{
    public string name;
    public int attackIndex;
    [Range(0, 100)] public float weight;
    public int minPhase = 1;
}

[Header("--- REFERENCES ---")]
public Transform player;
public BetterWeapon weaponScript;
private BossMovement movement;
private BossCombat combat;
private Animator anim;
private BossLightningAbility lightning;
private SekiroStats sekiroStats;

[Header("--- COMBO WEIGHT SYSTEM ---")]
public List<AttackConfig> attackConfigs = new List<AttackConfig>();

[Header("--- SETTINGS ---")]
public float attackRange = 2.2f;
public float combatCooldown = 3.0f;
private float lastAttackFinishTime;

[Header("--- LIGHTNING SKILL CONFIG ---")]
public float lightningPrepDistance = 6.0f;
public float maxPrepTime = 1.0f;
private float currentPrepTimer;

```

Figure 4.3.2-1: BossAI FSM for Combat and Skill Management

The next code segment to be analyzed is the BossAI script, which serves as the brain controlling the complex behaviors of the game's primary antagonist. This script manages a Finite State Machine (FSM) to transition flexibly between actions such as chasing, attacking, resting, and using special skills. Below is a detailed analysis of the three most important functions that define the Boss's intelligence and tactical nature.


```

int PickWeightedAttackIndex()
{
    List<AttackConfig> validAttacks = new List<AttackConfig>();
    float totalWeight = 0;

    foreach (var attack in attackConfigs)
    {
        if (sekiroStats.currentPhase <= attack.minPhase)
        {
            validAttacks.Add(attack);
            totalWeight += attack.weight;
        }
    }

    if (validAttacks.Count == 0) return -1;

    float randomValue = Random.Range(0, totalWeight);
    float currentSum = 0;

    foreach (var attack in validAttacks)
    {
        currentSum += attack.weight;
        if (randomValue <= currentSum) return attack.attackIndex;
    }

    return validAttacks[0].attackIndex;
}

```

Figure 4.3.2:- BossAI Weighted Attack Decision System

The first function, which best demonstrates the AI's controlled randomness, is `PickWeightedAttackIndex`. Rather than selecting attacks completely at random—which would make the fight either chaotic or predictable—this function employs a Weighted Random selection algorithm. First, it filters a list of valid attacks based on the Boss's current Phase, ensuring that more powerful abilities only appear once the Boss has lost a certain amount of health or transitioned into the second stage. Subsequently, the function calculates the total weight of all valid moves and picks a random value within that range to determine the next strike. This mechanism allows developers to fine-tune the frequency of each attack, ensuring the Boss prioritizes signature moves while still maintaining an element of surprise for the player.

```

1 reference
void HandleCooldown()
{
    movement.RotateTowards(player.position, 8f);
    float distance = Vector3.Distance(transform.position, player.position);

    if (distance > runToPlayerDistance)
    {
        movement.MoveTo(player.position, movement.runSpeed, 1.0f);
        return ;
    }
    else if (distance < minSafeDistance)
    {
        Vector3 retreatDir = (transform.position - player.position).normalized;
        movement.MoveTo(transform.position + retreatDir * 2f, movement.walkSpeed, -0.5f);
    }
    else if (distance < strafeDistance)
    {
        strafeTimer += Time.deltaTime;
        if (strafeTimer > 2.0f) { strafeDirection *= -1; strafeTimer = 0; }
        Vector3 sideDir = Vector3.Cross((player.position - transform.position).normalized, Vector3.up) * strafeDirection;
        movement.Strafe(sideDir, strafeDirection * 0.5f);
    }
    else
    {
        movement.MoveTo(player.position, movement.walkSpeed, 0.5f);
    }

    if (Time.time >= lastAttackFinishTime + combatCooldown) currentState = BossState.Chasing;
}

```

Figure 4.3.3:- Intelligent Positioning Logic in HandleCooldown

The second highly tactical function is HandleCooldown. In typical action games, cooldown periods are usually when the enemy stands still, providing players with a free opportunity to attack. However, in this project, the HandleCooldown function transforms this rest period into a strategic movement phase. The function continuously monitors the distance between the Boss and the player to make intelligent movement decisions: if the player is too far away, the Boss will run closer to maintain pressure; if the player is too close (within an unsafe zone), the Boss will back away to create space; and if at a medium distance, the Boss will strafe horizontally to probe and evade long-range attacks. This gives the impression of a true gladiator maneuvering around an opponent rather than a machine that only knows how to charge and strike.

```

2 references
public void OnHitByPlayer(float damageAmount, float incomingPostureDamage)
{
    if (combat == null || sekiroStats == null) return;

    // Check các điều kiện Stagger/Dead/PhaseTransition
    if (sekiroStats.isStaggered || sekiroStats.currentHP <= 0 || currentState == BossState.PhaseTransition) return;

    if (currentState == BossState.Cooldown || currentState == BossState.Chasing)
    {
        // Khi Boss đang thủ/chờ: Có thể truyền cả posture vào đây nếu BossCombat hỗ trợ
        // Hiện tại tạm giữ nguyên damageAmount, hoặc bạn có thể overload thêm
        combat.ProcessTacticalDefense(damageAmount);
    }
    else
    {
        // Khi Boss đang tấn công (Ăn đòn trực diện):
        combat.TakeDamageOnly(damageAmount);

        // --- SỬA Ở ĐÂY ---
        // Thay số cứng (10f, 30f) bằng tham số truyền vào từ vũ khí
        sekiroStats.ApplyDamage(damageAmount, incomingPostureDamage, false, false);
        // -----
    }
}

```

Figure 4.3.4-: Boss Combat Reaction Flow When Hit by Player

The third function is `OnHitByPlayer`, which is responsible for handling the Boss's reaction when attacked by the player. This function does more than just subtract health; it plays a decisive role in determining how the Boss reacts based on its current state. First, it checks for special conditions—such as whether the Boss is staggered, dead, or in the middle of a phase transition—to prevent logical errors. A standout feature of this function is its behavioral branching: if the Boss is in a resting or chasing state, it can trigger a tactical defense mechanism via the Combat Script; conversely, if the Boss is currently performing an attack, it will accept taking direct damage (Take Damage Only) in exchange for completing its strike. This mechanism creates the "Hyper Armor" feature commonly found in high-difficulty games, forcing players to think carefully before deciding to trade blows with the Boss.

4.3.3. Resurrection and Vitality Management System


```

void PerformRevive()
{
    if (deathCoroutine != null)
    {
        StopCoroutine(deathCoroutine);
        deathCoroutine = null;
    }

    canReviveInput = false;
    isDead = false;
    isDeadProcessed = false;

    isInvulnerable = true;

    // --- SỬA LỖI: BẬT LẠI THANH MÁU ---
    if (hpBarMain != null && hpBarMain.transform.parent != null)
    {
        hpBarMain.transform.parent.gameObject.SetActive(true);
    }
    // -----

    currentHP = maxHP * reviveHPPercent;
    currentPosture = 0;
    UpdateUI(); // Cập nhật lại thanh máu để nó hiện đúng % máu hồi

    if (UIManager.Instance != null) UIManager.Instance.HideDeathPanel();

    Animator anim = GetComponent<Animator>();
    if (anim != null)
    {
        anim.speed = 1f;
        anim.SetBool("isDead", false);
        anim.SetTrigger("Revive");
    }
}

```

Figure 4.3.2-2: PerformRevive

The PerformRevive function is the centerpiece of the "Second Chance" mechanic, allowing the player to return to the battle after being defeated once. When triggered, this function executes a sequence of actions to restore the character's combat state. First, it stops the death processing Coroutine (deathCoroutine) to prevent the transition to the Game Over screen. It then resets critical state flags: disabling the isDead variable to confirm the character is alive, disabling canReviveInput to prevent revival key spamming, and enabling isInvulnerable to provide a brief window of invincibility, ensuring the player is not immediately attacked upon standing up.

In terms of attributes, the PerformRevive function calculates the restored health based on a pre-configured percentage (reviveHPPercent), ensuring the player returns with enough health to continue fighting while still needing to remain cautious. The Posture bar is also reset to zero so the player can defend themselves immediately. Visually, the function reactivates the health bar on the User Interface (UI) if it was previously disabled and calls the HideDeathPanel function from the UIManager to hide the death notification. Finally, it signals the Animator to play the "Revive" animation and restores the animation speed to normal, returning the character to a controllable state.

```

public void ApplyDamage(float hpDamage, float postureDamage, bool isBlocked, bool isParried)
{
    if (isDead || isInvulnerable) return;

    if (isStaggered)
    {
        // Nếu là Boss: Bắt từ khi choáng (để bảo vệ logic chuyển phase)
        if (isBoss) return;

        // Nếu là Player: Vẫn đi tiếp xuống dưới để trừ máu (để kích hoạt cơ chế tỉnh lại)
    }

    float finalHPDamage = hpDamage;
    float finalPostureDamage = postureDamage;

    if (isParried) { finalHPDamage = 0; finalPostureDamage *= 0.1f; }
    else if (isBlocked) { finalHPDamage = 0f; finalPostureDamage *= 0.6f; }

    if (finalHPDamage > 0)
    {
        PlayRandomDamageSound();
    }

    currentHP -= finalHPDamage;
    if (currentHP <= 0)
    {
        Die();
        return;
    }
    if (!isStaggered)
    {
        currentPosture += finalPostureDamage;
        if (currentPosture >= maxPosture) BreakPosture();
    }

    UpdateUI();
}

```

Figure 4.3.2-3: ApplyDamage

The ApplyDamage function handles the general damage processing logic, applicable to both Health and Posture. This function accepts input parameters including health damage (hpDamage), posture damage (postureDamage), and defensive states such as isBlocked (normal block) or isParried (perfect parry). Based on these parameters, the function calculates the actual damage the character sustains.

If the attack is Parried, health damage is completely negated to zero, and damage to Posture is reduced to a minimal amount (10%), rewarding the player's skill. If the attack is only Blocked, health damage is still blocked, but the damage to Posture is significantly higher (60%), putting pressure on the player's defensive capacity. If the character fails to defend, they receive full damage to both Health and Posture.

Following the calculations, the function subtracts the corresponding health from the currentHP variable. If health drops to or below zero, the function calls the Die method to handle death. Simultaneously, the function accumulates damage to the Posture bar (currentPosture). If this bar fills up (exceeding maxPosture) and the character is not already stunned, the function calls BreakPosture to put the character into a staggered state, opening an opportunity for the opponent to perform a finisher. Additionally, ApplyDamage integrates feedback elements such as playing injury sounds (PlayRandomDamageSound) and updating

the user interface (UpdateUI) to ensure the player immediately grasps the combat situation. This combination of logical calculation and visual feedback creates a tight and fair combat experience.

4.3.4. *Weapon Hit Detection and Damage Application Logic*

```
public class Weapon : MonoBehaviour
{
    [Header("Stats")]
    public float hpDamage = 10f;
    public float postureDamage = 30f; // Đã sửa mặc định thành 30 (h

    [Header("Points Reference")]
    public Transform tipPoint; // Vị trí mũi kiếm
    public Transform basePoint; // Vị trí chuôi kiếm
    public LayerMask hitLayer; // Layer của đối thủ (Player hoặc En

    [Header("Visuals")]
    public TrailRenderer trail;
    public GameObject owner;

    private bool isScanning = false;
    private List<GameObject> alreadyHit = new List<GameObject>();

    public void EnableTriggerBox()
    {
        isScanning = true;
        alreadyHit.Clear();
        if (trail) trail.emitting = true;
    }

    public void DisableTriggerBox()
    {
        isScanning = false;
        if (trail) trail.emitting = false;
    }
}
```

Figure 4.3.2-4:

The final code segment analyzed in the combat system is the Weapon script, which is responsible for handling physical collisions between the player's weapon and enemies, as well as transmitting damage parameters. Instead of using traditional Colliders, which can lack precision during high-speed action sequences, this script employs a raycasting method to ensure every slash is accurately recorded at the moment of contact.

The two functions `EnableTriggerBox` and `DisableTriggerBox` serve as switches to activate the damage mechanism and are called directly via Animation Events. When the character begins to swing their sword, `EnableTriggerBox` is invoked to set the `isScanning` flag to true, allowing the system to start detecting collisions. Simultaneously, it clears the `alreadyHit` list to ensure a new strike can register damage from scratch and activates the `TrailRenderer` visual effect to create an aesthetic sword trail. Conversely, when the slashing

motion concludes, DisableTriggerBox disables the isScanning variable and stops the trail effect, preventing unintended damage while the character sheathes their weapon. Furthermore, the EnableTrail and DisableTrail functions act as bridge functions, ensuring the script remains compatible with calls from the legacy combat system without requiring a complete overhaul of the Animation Events.

```
void ScanForHit()
{
    RaycastHit hit;
    // Kiểm tra va chạm bằng đường thẳng giữa 2 điểm Tip và Base
    if (Physics.Linecast(basePoint.position, tipPoint.position, out hit, hitLayer))
    {
        GameObject victim = hit.collider.gameObject;

        if (victim != owner && !alreadyHit.Contains(victim))
        {
            HandleHit(victim);
            alreadyHit.Add(victim);
        }
    }
}

void HandleHit(GameObject victim)
{
    // Kiểm tra nếu chủ nhân là Player đánh trúng Enemy
    if (owner.CompareTag("Player") && (victim.CompareTag("Enemy") || victim.CompareTag("Boss")))
    {
        // 1. Ưu tiên tìm script AI của Boss trước
        if (victim.TryGetComponent<BossAI>(out var bossAI))
        {
            // --- CẬP NHẬT QUAN TRỌNG TẠI ĐÂY ---
            // Truyền cả hpDamage và postureDamage sang cho BossAI xử lý
            bossAI.OnHitByPlayer(hpDamage, postureDamage);

            Debug.Log($"Player chém Boss! HP: {hpDamage}, Posture: {postureDamage}");
        }
        // 2. Nếu không phải Boss (quái thường), tìm Stats chung
        else if (victim.TryGetComponent<SekiStats>(out var genericStats))
        {
            // Quái thường nhận damage trực tiếp
            genericStats.ApplyDamage(hpDamage, postureDamage, false, false);
        }
    }
}
```

Figure 4.3.2-4: Boss combat system

The Update and ScanForHit functions are the heart of the collision detection system. In every frame, the Update function checks if the isScanning flag is active; if so, it calls the ScanForHit function. ScanForHit utilizes the Physics.Linecast method to draw an invisible line between the hilt (basePoint) and the tip of the blade (tipPoint). If this line intersects any object belonging to the enemy layer (hitLayer), the system confirms it as a valid hit. This method overcomes the disadvantages of a Box Collider, which can sometimes pass through an enemy without registration during low frame rates or high-speed attacks. Once a collision is identified, the function verifies that the victim is not the weapon's owner and has not already been hit during the current swing; if these conditions are met, it triggers the hit-handling logic and adds the victim to the hit list.

Finally, the HandleHit function is responsible for categorizing and applying damage to the target. This function executes a hierarchical check to ensure game logic functions correctly

for different types of enemies. First, it verifies whether the weapon wielder is the Player and if the target is an Enemy or a Boss. The script then prioritizes searching for a BossAI component on the target. If found, signifying the target is a Boss, the system calls the BossAI's OnHitByPlayer function, passing both health damage (hpDamage) and posture damage (postureDamage) so the AI can process complex reactions like blocking or counter-attacking. If no BossAI is found, the script looks for a SekiroStats component to treat the target as a standard mob, applying damage directly to its stats. This design separates the processing logic between bosses and regular minions while ensuring the combat system remains highly scalable.

4.3.5. Save game

```
// --- HÀM LƯU GAME ---
public void SaveGame()
{
    string currentScene = SceneManager.GetActiveScene().name;

    // 1. Lưu dữ liệu
    PlayerPrefs.SetString("SavedScene", currentScene);
    PlayerPrefs.SetFloat("PosX", transform.position.x);
    PlayerPrefs.SetFloat("PosY", transform.position.y);
    PlayerPrefs.SetFloat("PosZ", transform.position.z);

    // 2. Ghi xuống ổ cứng ngay lập tức (Quan trọng để không bị mất khi tắt game đột ngột)
    PlayerPrefs.Save();

    // In ra màn hình console để bạn biết là nó ĐANG CHẠY
    Debug.Log($"✅ [SAVE] Đã lưu thành công! Scene: {currentScene} tại vị trí {transform.position}");
}

// --- HÀM TẢI GAME ---
void LoadGamePosition()
{
    // Reset biến này về false ngay lập tức để tránh lỗi cho lần sau
    GameSession.isLoadingGame = false;

    if (PlayerPrefs.HasKey("PosX"))
    {
        float x = PlayerPrefs.GetFloat("PosX");
        float y = PlayerPrefs.GetFloat("PosY");
        float z = PlayerPrefs.GetFloat("PosZ");
        Vector3 loadedPos = new Vector3(x, y, z);

        // Mẹo: Tắt CharacterController đi 1 nhịp để dịch chuyển không bị kẹt
        if (characterController != null) characterController.enabled = false;

        transform.position = loadedPos;
    }
}
```

Figure Error: Reference source not found-31: Saving and Loading Game State Using PlayerPrefs

This code implements a game progress persistence mechanism that allows the system to store and restore the player's state between play sessions. The current scene name and the player's world position are saved to persistent storage, ensuring that progress is not lost even if the game is closed unexpectedly. When the game is loaded, the saved data is retrieved and applied to restore the player's exact position within the scene. To maintain stability and avoid physics-related issues, character movement control is temporarily

4.4. Screenshots of main features



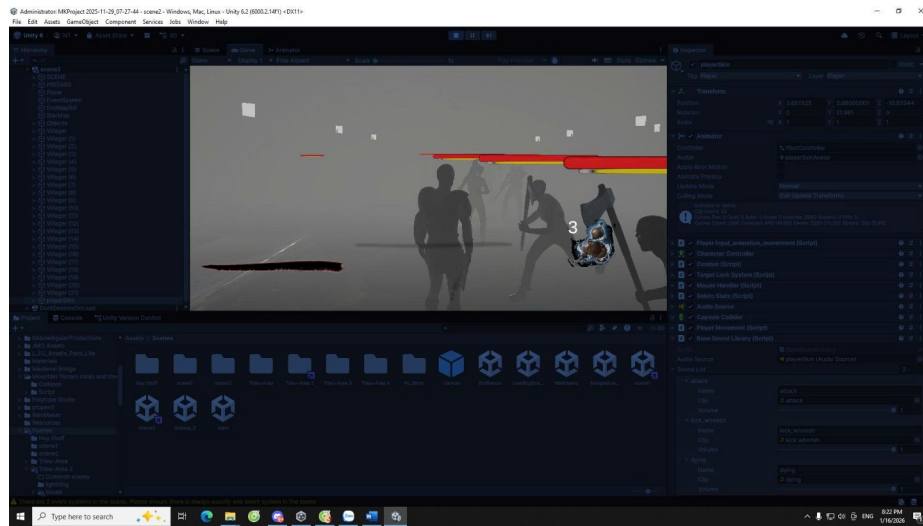


Figure Error: Reference source not found-35: Enemy Detection System



Figure Error: Reference source not found-36: Player Healing System

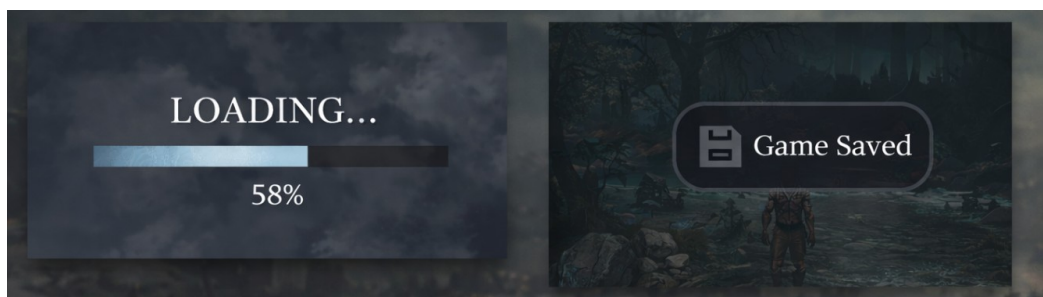


Figure Error: Reference source not found-37: Saving/Loading states Screen

4.5. Conclusion and future work

This project successfully demonstrates the development of a single-player action game prototype that emphasizes skill-based combat and responsive gameplay mechanics. By integrating character control, collision detection, scoring, and progression systems within a

unified framework, the game delivers a coherent and engaging experience. The project also serves as a practical application of game development principles, including real-time interaction handling, system integration, and user interface design. Overall, the completed prototype meets its initial objectives and provides a solid foundation for further expansion and refinement.

4.5.1. Achievements

Several key objectives were achieved during the development of this project. A responsive character control system was implemented, allowing precise movement, jumping, and combat actions. Collision detection and interaction handling ensure realistic responses between the player character, enemies, and environmental elements such as platforms and obstacles. A score system was successfully designed and integrated, evaluating player performance based on combat outcomes and enabling level progression. Additionally, game progress and state transitions were implemented to allow smooth advancement between gameplay, score evaluation, and story-related content, contributing to a structured and immersive game flow.

4.5.2. Limitations

Despite its successful implementation, the game has several limitations. The current version is limited to a single-player experience and does not support multiplayer functionality or online interaction. Enemy behavior is relatively basic and lacks advanced artificial intelligence features such as adaptive strategies or learning-based responses. In addition, the scope of the game content is restricted, with a limited number of levels, enemy types, and environmental variations. Technical constraints and time limitations during development also restricted the implementation of advanced visual effects and optimization features.

4.5.3. Suggestions for future enhancements

Future development could focus on expanding both gameplay depth and technical features. Additional levels with varied environments and more complex level design could enhance replayability. Combat mechanics could be extended through new abilities, power-ups, and enemy types to increase strategic diversity. Improvements to graphics, animations, and visual effects would further enhance immersion. From a system perspective, implementing online leaderboards or achievement tracking could encourage competition and

long-term engagement. In the long term, introducing more advanced enemy AI or optional multiplayer features could significantly enrich the overall game experience.

APPENDIX

Appendix 1. Project source code (Google drive)

Google drive link: [MKProject 2025-11-29_07-27-44.zip - Google Drive](#)

Description: This repository contains the full source code of the project, including backend, frontend, and configuration files.

Appendix 2. AI/Tools usage statement

Tools used: Visual Studio, Unity, Unity Hub, Devops version control unity

Generative AI:

Used for: summarizing requirements, suggesting naming conventions, improving English grammar in report, code guidance.

Not used for: direct copy-paste of full modules without review.

Verification: manual testing + team review.

Appendix 3. Unity Version Control

Unity version control link : [Unity Cloud](#)

Description: Unity Version Control (Unity Cloud) is used to manage the game project's source code and assets throughout development. It provides centralized version control, allowing changes to be tracked, committed, and reviewed over time. The system supports collaborative workflows through branching and merging, ensuring that multiple contributors can work in parallel without data loss. In addition, Unity Version Control efficiently handles large binary assets such as scenes, textures, models, and audio files, making it well-suited for Unity-based game development projects.

REFERENCES

Unity Technologies. (n.d.). Unity Hub documentation.

Retrieved January 2026, from

<https://docs.unity3d.com/hub/manual/index.html>

Unity Technologies. (2024, August 20). Install the Unity Editor using Unity Hub.

Retrieved January 2026, from

<https://docs.unity3d.com/hub/manual/InstallEditors.html>

Microsoft Learn. (2024, October 10). Game development using Visual Studio 2022.

Retrieved January 2026, from

<https://learn.microsoft.com/en-us/visualstudio/gamedev/>

Microsoft Learn. (n.d.). Visual Studio 2022 documentation.

Retrieved January 2026, from

<https://learn.microsoft.com/en-us/visualstudio/>

TASK DIVISION AND CONTRIBUTION SUMARY

Member name	Assigned tasks	Workload contribution (%)	Completion status (%)
Nguyễn Ngọc Triệu	Designed Game UI, built maps, sourced map assets, wrote reports, and developed main scripts for boss	16,6%	100%
Hoàng Văn Thiên	Wrote reports, designed UI	16,6%	100%
Nguyễn Hoàng Phúc	Wrote reports and created PowerPoint presentations	16,6%	100%
Nguyễn Thành Đạt	Designed menus, wrote word	16,6%	100%
Nguyễn Tấn Thiện Huy	Designed menus, edited sound effect and built maps	16,6%	100%
Phạm Quang Vũ	Developed main scripts for characters and levels, wrote reports	16,6%	100%